

Desarrollo Aplicaciones

2r.SMX – Apuntes asignatura 0.2

HTML

[01 – Clases de etiquetas](#)

[02 – Etiquetas Semanticas](#)

[03 – Configuracion del Head](#)

[04 – Adiccion de selectores](#)

[05 – Acceso a etiqueta sin selector](#)

[06 – Atributos](#)

[07 – Etiquetas de formato texto](#)

[08 – Etiquetas de listado](#)

[09 – Etiquetas de enlace](#)

[10 – Etiquetas de imagen](#)

[11 – Etiqueta de tabla](#)

[12 – Propiedades CSS Tablas](#)

[13 – Etiquetas de formulario](#)

[14 – Etiquetas métodos adicionales](#)

- Editar contenido de contenedor a tiempo real
- Input matriz de color a tiempo real
- Marcar textos, palabras o zonas de una pagina HTML
- Barra de progreso porcentual
- Creacion de mensajes emergentes desde HTML5

CSS

[15 – Selectores de uso común](#)

[16 – Uso de fuentes personalizadas](#)

[17 – Seudo selectores](#)

[18 – Seudo elementos](#)

[19 – CSS Flex](#)

[19.3 – Declaracion de contenedores Flex](#)

[19.4 – Declaracion de direccionamiento de elementos Flex](#) [Row]

[19.5 – Declaracion de direccionamiento de elementos Flex](#) [Colum]

[19.6 Declaracion de direccionamiento de elementos Flex](#) [\[Reverses\]](#)

[19.7 Ajuste de elementos Flex](#)

19.8 Alineacion horizontal de elementos Flex

[19.9 Alineacion horizontal de elementos Flex](#) [\[Justify-content\]](#)

[19.10 Alineacion vertical de elementos Flex](#) [\[Align-items\]](#)

[19.11 Alineacion horizontal wrap Flex](#) [\[Justifi content\]](#)

[19.12 Orden de aparición de elementos Flex](#)

[19.13 Deformacion de elementos Flex](#)

[20 - Alineacion individual en grupos de componentes Flex](#)

[21 – Metodos de posicionamiento de elementos HTML](#)

[21.1 Position : absolute](#)

[21.2 Position :relative](#)

[21.3 Position:fixed](#)

[21.4 Position:sticky](#)

Anexo_HTML

[00-Eliminar efectos en href](#)

01- [Dejar fijo el background de una website](#)

02- [Ocultar mostrar elemento html](#)

03- [Modificar transparencia de un elemento](#)

[22 – Grid](#)

[22.1 Definicion de contenedor Grid](#)

[22.2 Propiedades del Grid](#) [\[Definir columnas y filas\]](#)

[22.2 Propiedades del Grid](#) [\[Separacion entre columnas y filas\]](#)

[22.3 Definir correspondencia area](#) [\[Asignar áreas a elementos\]](#)

[22.4 Definir forma del grid](#) [\[Asignar áreas a zonas del grid\]](#)

[22.5 Metodo alternativo de asignación de áreas al grid](#)

[22.6 Anexo](#) [\[Ejemplo de grid\]](#)

CSS

[23 - Adaptative Web](#) [\[Concepto Viewport\]](#)

[23.1 Breakpoint](#) [\[Puntos de ruptura\]](#)

[23.2 Media Queries](#) [\[@media\]](#)

[23.2 Media Queries](#) [\[Declaracion\]](#)

[23.3 Metodo de trabajo en Adaptative Web](#)

[23.4 Adaptacion de imágenes en Adaptative Web](#)

[23.5 Uso de tamaños de letra adaptativa](#)

JS

[24 - Inclusion de Java Scrit en documentos HTML5](#)

[25 – Operadores en Java Script](#)

[25.1 Operadores en Java Script](#) [\[Adicion resta\]](#)

[25.2 Operadores en Java Script](#) [\[Asig.add - Asig.Resta\]](#)

[25.3 Operadores en Java Script](#) [\[Operadores de comparacion\]](#)

[25.4 Operadores en Java Script](#) [\[Operadores Aritmeticos\]](#)

[25.5 Operadores em Java Script](#) [\[Operadores logicos\]](#)

[25.6 Operadores en Java Script](#) [\[Condicional ternario\]](#)

[25.7 Resto de operadores en Java Script](#)

[26 – Tipos de declaración de datos](#)

[27 – Metodo Array en Java Script](#)

[28 – Bucles](#) [\[Basicos\]](#)

[29 – Condicionales / Switches](#)

[30 – Funciones](#) [\[Basicas\]](#)

[31 – Objetos](#) [\[Avanzado\]](#)

[32 – Metodo de trabajo con objetos](#) [\[Constructors\]](#)

[33 – Metodos de trabajo DOM](#)

[33.1 Metodo DOM](#) [\[HTML\]](#)

[33.2 Sintaxis del método DOM](#) [\[id - class\]](#)

[33.3 Sintaxis del método DOM](#) [\[Styles\]](#)

[33.4 Modificar código HML por DOM](#)

[33.5 Metodo DOM](#) [\[innerHTML\]](#)

Anexo material de exámenes

[34 – Numeros aleatorios en Java Script](#) [\[Material de examen\]](#)

[35 – Transformar variable en integer](#) [\[Material de examen\]](#)

[36.1 Declarar variables desde el comienzo](#) [\[Material de examen\]](#)

[36.2 Transformar strings en números enteros](#) [\[Material de examen\]](#)

[37 – Verificacion de datos en inputs](#) [\[Material de examen\]](#)

[37.1 Metodo de comprobación](#) [\[typeof\(\)\]](#)

[37.2 Metodo de comprobación](#) [\[!isNaN\(\)\]](#)

[38 – Verificacion de la longitud de un dato](#) [\[Material de examen\]](#)

[39 – Transformar variable en string](#) [\[Material de examen\]](#)

[40 – Dividir un string](#) [\[Material de examen\]](#)

[41 – Dividir un string generando un array](#) [\[Material de examen\]](#)

[42 – Localizar un carácter en string](#) [\[Material de examen\]](#)

[43 – Funcion que calcula la media de dos números](#) [\[Examen 03\]](#)

[44 – Funcion que calcula el iva de un producto](#) [\[Examen 03\]](#)

[45 – Funcion que calcula totales según requerimientos](#) [\[Examen 03\]](#)

[46 – Funcion que calcula totales según requerimientos \(02\)](#) [\[Examen 03\]](#)

[47 – Funcion que calcula totales según requerimientos \(03\)](#) [\[Examen 03\]](#)

[48 – Funcion con validación de carácter en string](#) [\[Examen 03\]](#)

[49 – Funcion calculo mediana mediante array estaico](#) [\[Examen 03\]](#)

[50 – Funcion comparación de fechas](#) [\[Examen 03\]](#)

[51 – Funcion con reconocimiento string y conversión del mismo](#) [\[Examen 03\]](#)

[52 – Funcion con reconocimiento e iteración condicionado a string](#) [\[Examen 03\]](#)

[53 – Funcion con calculo condicionado a valor de dato](#) [\[Examen 03\]](#)

[56 – Ocultar elemento en HTML, activándolo por Java Script en DOM](#) [\[Examen 03\]](#)

Anexo material de exámenes

[57 – Añadir parámetros condicionales en IF](#) [\[Examen 03\]](#)

[58 – Hacer no editable un input](#) [\[Examen 03\]](#)

[59 – Variables todo a mayúsculas – minúsculas](#) [\[Examen 03\]](#)

Anexo 02

[60 - Metodos avanzados con arrays](#)

[60.1 Metodo](#) [\[Filter\]](#)

[60.2 Metodo](#) [\[find\]](#)

[60.3 Metodo](#) [\[includes\]](#)

[60.4 Metodo](#) [\[indexOf\]](#)

[61 – Metodos de trabajo JSON](#)

[61.1 Creacion del .json](#)

[61.2 Estructura de datos del .json](#)

[61.3 Carga del .json](#) [\[En archivo de texto\]](#)

[61.4 Carga del .json](#) [\[En variables\]](#)

[61.5 Carga de objeto a .json](#) [\[Inversa\]](#)

[62 – Metodos de trabajo Ajax](#)

[62.1 Carga de datos .json en Ajax](#) [\[Sincronico\]](#)

[62.2 Datos Ajax sincronico para DOM Java Script](#)

[62.3 Filtro de carga mediante dropdown HTML](#)

[62.4 Actualizacion de carga del src .js](#)

[62.5 Actualizacion de sistema de datos en json](#)

[63 – Metodo Promise Javascript](#) [\[Ultimo anexo\]](#)

[64 – Metodo DOM producto de constructor y objetos](#) [\[Penultimo examen\]](#)

[11-10-23 20:20]

Atención:

Los siguientes apuntes de codificación web corresponden a la versión de HTML 5 en vigor en octubre del 2022. Los métodos, semánticas y selectores están en constante modificación y transformación, por lo que antes de emplear esta información, verifique la versión de HTML que maneja y los cambios acaecidos en referencia a esta misma.

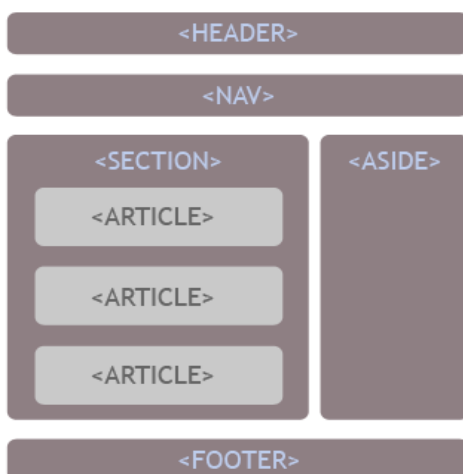
01- Clases de etiquetas:

HTML

Semánticas	Definen áreas y sectores del documento HTML, se emplean posteriormente como referencia en la programación Adaptative y en los scripts JavaScript y PHP.
Atributos	Son las sub-etiquetas que están en el interior de las semánticas. Los atributos siempre están referenciados en la etiqueta de apertura.
Selectores	Son las etiquetas que hacen referencia a clases o referencias. Se emplean únicamente en el CSS.
Sub-Selectores	Son etiquetas que se emplean en combinación con los selectores, suelen ir tras ellos. Solo se emplean en CSS.

02 – Clases de etiquetas:**[Semánticas]**

Existe una amplísima variedad de etiquetas semánticas. Cada versión de HTML nueva suele acrecentar su número.



Esta ilustración muestra una configuración básica de un documento .HTML

```
<header> </header>
```

Cabecera del documento, lleva links e información de codificación.

```
<body></body>
```

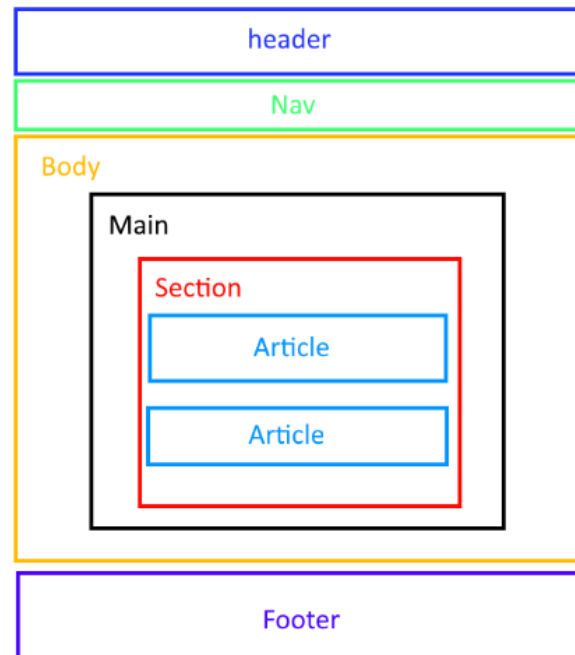
Cuerpo central del documento.

```
<footer></footer>
```

Pie del documento.

02 – Clases de etiquetas:

[Semánticas]

`<header> </header>``<nav></nav>``<body>``<main>``<section>``<article> </article>``<article> </article>``</section>``</main>``</body>``<footer></footer>`**Atención:**

Estas no son todas las etiquetas semánticas que existen, solo una brevísima exposición de ellas.

Header	Contiene toda la información referente a la codificación y links de CSS del documento .HTML .
Nav	Etiqueta semántica que define un sector a modo de “navegador” tiene sub-atributos de background y border.
Body	Semántica que define el área principal del documento HTML. Se le pueden atribuir selectores, posee numerosos sub-atributos, entre los que destacan Background-image.
Main	Semántica que marca sub-áreas dentro del body. Se le pueden atribuir selectores. Al igual que Body posee numerosos sub-atributos. *Detalle de <i>background-image</i> de main: Solo es visible cuando se le añade contenido.
Section	Define áreas en el interior del body – main. Al igual que los anteriores dispone de numerosos sub-atributos y se le pueden definir selectores.
Article	Sub-secciones dentro de la semántica section. Poseen numerosos sub-atributos y también se le pueden definir selectores.
Footer	Área de pie de página. Comparte sub-atributos y selectores con el resto de semánticas del documento.

02 – Clases de etiquetas:**[Clases de etiquetas]****Etiquetas con cierre**

Son todas las etiquetas que requieren de un cierre `</>` para poder ser efectivas.

Comunes:

- Todas las semánticas de base.
- Todas las de relacionadas con estilo, tablas, href.
- Todas las que definen secciones.

Etiquetas auto-conclusivas

Son las etiquetas que no requieren de cierre para poder ser efectivas. Aun así, estas tienen la barra de cierre al final de su instrucción.

Comunes:

```
<img src="" />
<br>
<link href="" rel="" type="" />
```

03- Configuración del Head**[Configuración en AW 2022]**

*iniciar la codificación: html5 – tab

<!--Esto es el área de los Link correspondientes a los CSS -->

```
<link href="css/normalize.css" rel="stylesheet" type="text/css" />
```

```
<link href="css/style.css" rel="stylesheet" type="text/css" />
```

<!--Esto es el área de los Link correspondientes al icono de la pagina -->

<!--El icono es una imagen con transparencia de 32x32 pixeles, PNG, modificado su extensión a .ICO -->

```
<link rel="shortcut icon" href="img/icono.ico">
```

<!--Aquí abajo es donde están las Fuentes referenciadas del Google font -->

```
<link rel="preconnect" href="https://fonts.googleapis.com">
```

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

```
<link href="https://fonts.googleapis.com/css2?family=Roboto+Mono:wght@100&display=swap" rel="stylesheet">
```

```
<!-- #####
```

```
| Santiago García Santiago |
```

```
| AW - 2. SMX [00-10-2022]|
```

```
##### -->
```

04 – Clases de etiquetas:**[Añadir selectores]**

La mayor parte de las etiquetas poseen una serie de atributos inherentes a su acción. Así mismo podemos modificar el comportamiento de estas etiquetas (o añadirle nuevas funcionalidades) desde el CCS, mediante el empleo de selectores. Se pueden añadir selectores tanto a semánticas como a etiquetas.

Existen dos clases de selectores:**Selector de clase****Se define añadiendo a la etiqueta de apertura;**

Class="id" Donde Id corresponde al nombre que le asignamos.

```
<table class="ejemplo_01" > </table>
```

Acceso mediante CSS:

```
.ejemplo_01 {  
    border: 1px solid black;  
}
```

Selector de id**Se define añadiendo a la etiqueta de apertura;**

id="id" Donde Id corresponde al nombre que le asignamos.

```
<table id="ejemplo_02" > </table>
```

Acceso mediante CSS:

```
#ejemplo_02 {  
    border: 1px solid black;  
}
```

Class:

Se accede a la etiqueta referenciada en la clase, situando un punto delante del id.

Id:

Se accede a la etiqueta referenciada en el id, situando una almohadilla delante del id.

04 – Clases de etiquetas:**[Añadir múltiples selectores]**

Se pueden asignar más una clase simultáneamente a una misma semántica o etiqueta. Para ello simplemente tenemos que separar cada una de estas clases o id por un espacio en blanco.

Ejemplo; asignación de una clase común y una única por cada uno de los article de section:

```
<main>
<section class="contenedor">

    <article class=" centro art_01">
        </article>
    <article class=" centro art_02">
        </article>

</section>

</main>
```

En este caso todos los artículos del código tienen como clase "común":

[centro]

Y como elemento diferencial:

[art_01]

[art_02]

Por lo que todo que definiéramos en el CSS en el selector [Centro] se ejecutaría de igual modo en todos los article. Pero cada uno de estos article, podría tener elementos CSS diferentes mediante su selector único.

05 – Clases de etiquetas:**[Accediendo a etiqueta sin selector]**

Se puede acceder a una etiqueta o semántica sin necesidad de asignarle previamente un selector, desde el propio CSS.

Ejemplo:

```
<main>
<section class="contenedor">

    <article class=" centro art_01">
        <table>
            <tr> <th></th> </tr>
        </table>
    </article>

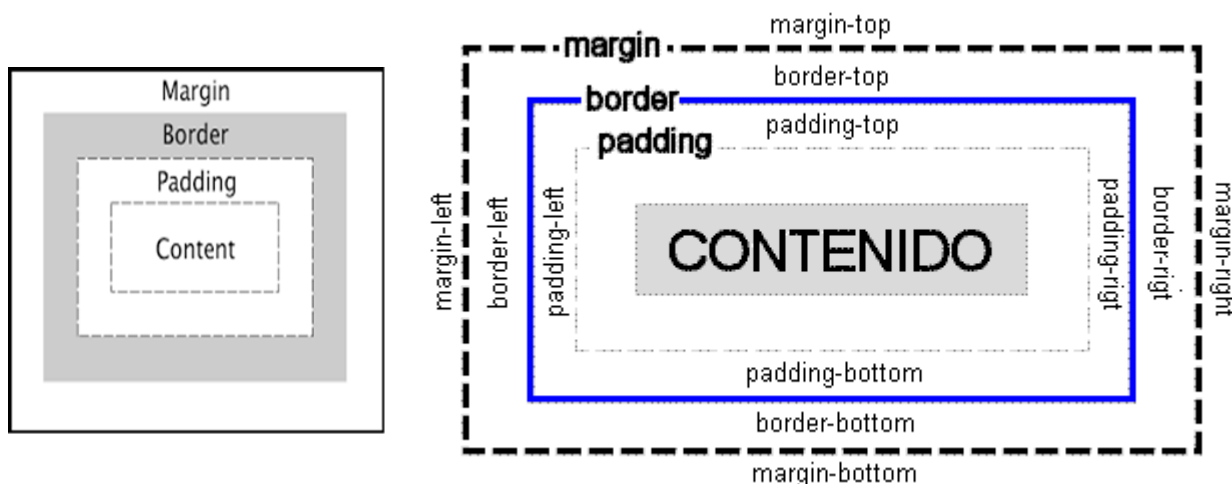
</section>

</main>
```

Podríamos acceder a la tabla en CSS mediante este código:

```
section table {
border: 1px solid black;
}
```

Simplemente ponemos el nombre de la etiqueta que queremos seleccionar tras la etiqueta principal, seleccionando así uno de los "hijos" de la principal.

06 – Clases de atributos:**[Margin, padding, border]**

Todas estas propiedades pueden ser expresadas en porcentajes o pixeles (% , px).

Si se define el margin como 0px el elemento se centra horizontalmente.

Ejemplo:

```
td {
```

```
Margin-top: 15px;
```

```
Padding-left: 5px;
```

```
}
```

Margin	Corresponde a los márgenes alrededor del objeto. Si se define a 0px, se centra horizontalmente.
Border	Corresponde a los bordes del objeto, si se quieren hacer visibles, se tiene que añadir "solid" y un color en la misma línea.
Padding	Es un relleno transparente entre el objeto y lo que le rodea.

07 – Etiquetas de formato de texto:**Títulos**

Los títulos están divididos del `<h1></h1>` al `<h6></h6>` se pueden modificar sus estilos, referenciándolos en el CSS.

Propiedades especiales:

Los títulos, pueden tener un color de fondo, o incluso una imagen.

Ejemplo:

```
h1 {
```

```
background-image: url("../img/fondo_h.png");
```

```
background-repeat: no-repeat;
```

```
background-position: left;
```

```
background-size: 28px;
```

```
}
```

07 – Etiquetas de formato de texto:**[Continuación]****Formato negrita**

Se puede emplear desde el HTML o desde el CSS. Desde el html solo hay que poner el texto en el interior de esta etiqueta:

```
<b>aquí el texto</b>
```

Y en el css:

```
Table tr:nth-child(1){
    font-weight: bold;
}
```

Párrafos

Podemos indicar la existencia de párrafos o bloques de texto (se comportan como los DIV, apilación vertical) colocando el bloque de texto entre la etiqueta:

```
<p> aquí el bloque de texto</p>
```

Propiedades especiales:

Se puede emplear para facilitar el centrado de una imagen o contenedor de imagen.

Ejemplo:

```
<section>
```

```
<article>
```

```
<p> <p>
```

```
</article>
```

```
</section>
```

Para centrar la imagen solo, tendríamos que seleccionar “article” en CSS y asignarle “text-align:center;”

Salto de línea

Para introducir saltos de línea entre textos o elementos, solo tenemos que introducir la etiqueta auto-conclusiva:

```
<br>
```

Separación de temas

Para separar “visiblemente” bloques o temas, tenemos que emplear la etiqueta auto-conclusiva:

```
<hr>
```

08 – Etiquetas de listado:

En HTML 5 podemos listar contenidos mediante dos tipos de listado;

Listas no numeradas

Estas listas muestran los elementos mediante tokens o iconos.

- i. General Aviation
 - a. Single Aviation Aircraft
 - b. Dual-Engine Aircraft
- ii. Commercial Aviation
 - Dual-engine
 - Tri-engine

- A
 - B
 - 1. C
 - 2. D
 - E
 - F
 - G
- B

Código:

```
<ul>
  <li>elemento_01</li>
  <li>elemento_02</li>
</ul>
```

Elegir los tokens:

```
<ul type="a"> </ul>      Orden por letras.
<ul type="i"> </ul>      Orden por números romanos.
<ul type="disc"> </ul>   Orden por discos.
<ul type="square"> </ul> Orden por cuadrados.
```

Listas numeradas

Estas listas muestran los elementos mediante un sistema numerado.

Código:

```
<ol>
  <li>elemento_01</li>
  <li>elemento_02</li>
</ol>
```

08 – Etiquetas de listado:**[Continuación]****Listas desplegables [1]**

Podemos crear listas desplegables mediante la etiqueta “details”. Es importante recordar que cada elemento es una etiqueta individual.

Código:

```

► Programación backend
▼ Programación frontend
La programación frontend es para
tipos también bien buena onda

```

```
<details>
```

```
<summary> primer elemento desplegable</summary>
```

```
<p> aquí estaría el texto del primer elemento </p>
```

```
</details>
```

```
<details>
```

```
<summary> segundo elemento desplegable</summary>
```

```
<p> aquí estaría el texto del segundo elemento </p>
```

```
</details>
```

09 – Etiqueta de enlace:

En HTML 5 disponemos de tres tipos de enlace diferentes;

Enlace a sección de la página

Este enlace dirige al usuario a la sección del documento referenciada.

```
<a href="#contacto"> texto_o_imagen </a>
```

En este caso nos dirigiría a la sección con el Id="contacto".

Enlace a URL

Este enlace cargaría la URL indicada en la misma pestaña que estamos empleando.

```
<a href="google.com"> texto_o_imagen </a>
```

Enlace en otra pestaña

Podemos redirigir la carga a otra pestaña del navegador que estemos empleando. Para ello, solo tenemos que añadir el atributo “target=”_blank” a nuestro enlace.

```
<a href="Polisse.html" target="_blank">
```

10 – Etiqueta de imagen:

Las imágenes pueden mostrarse por sí mismas mediante una etiqueta, o bien como parte del “fondo” de algunas etiquetas (inc. Semánticas) o anidadas en contenedores del estilo DIV, td, th.

Código:

```

```

Las etiquetas de imagen pueden referenciarse en CSS directamente o mediante la asignación de selector de clase / id.

Ejemplo:

```
img {
```

```
Width=15px;
```

```
}
```

Si solo definimos una de sus propiedades (width o height) el navegador ajustara proporcionalmente la imagen.

Podemos seleccionar estas propiedades mediante pixeles o porcentajes.

[px , %]

11 – Etiqueta de tabla:

Las tablas son uno de los elementos más complejos y al mismo tiempo útiles de la codificación en HTML 5.

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>
```

```
</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<tr> <td>
```

```
</td>
```

```
</tr>
```

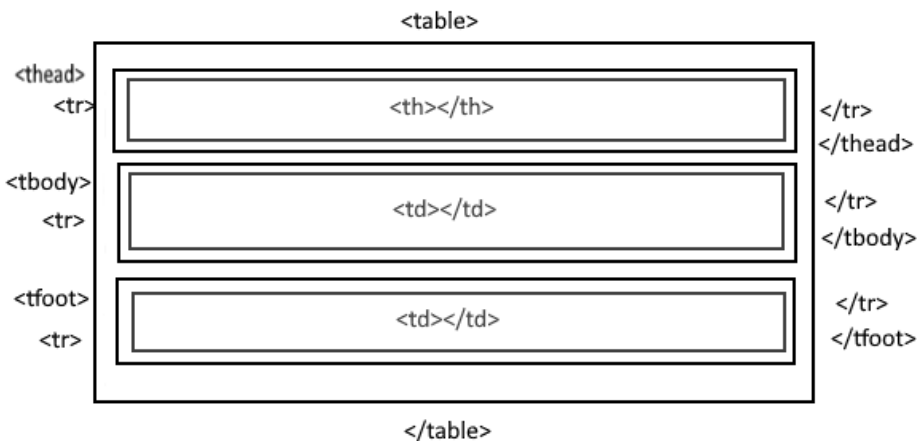
```
</tbody>
```

```
<tfoot>
```

```
<tr><td></td></tr>
```

```
</tfoot>
```

```
</table>
```



```
<tr></tr>
```

Define las filas de la tabla.

```
<td></td>
```

Define las columnas/celdas dentro de la fila.

```
<th></th>
```

Columnas/celdas de la primera fila.

```
<thead></thead>
```

Define la cabecera.

```
<tbody></tbody>
```

Define el centro de la tabla.

```
<tfoot></tfoot>
```

Define el final de la tabla.

11 – Etiqueta de tabla:**[atributos adicionales]**

Todos los elementos de las tablas, cuentan con una numerosa cantidad de atributos. Estos suelen ser configurados o definidos desde el Style.ccs, salvo el atributo de expansión de celdas;

Combinado de celdas en vertical

Mediante este atributo, expandimos el ancho de la celda en vertical.

Código:

```
<td colspan="4"> </td>
```

Esta celda ocuparía el espacio de 4 celdas en vertical.

Combinado de celdas en horizontal

Mediante este atributo, expandimos el ancho de la celda en horizontal.

Código:

```
<td rowspan="2"></td>
```

Esta celda ocuparía el espacio de 2 celdas en horizontal.

12 – Propiedades CSS etiqueta tabla:

La etiqueta de tabla y sub etiquetas poseen numerosos atributos que pueden ser manipulados desde el CSS.

Código:**Eliminar bordes entre celdas y filas**

```
Table{
    border-collapse:collapse;
}
```

Definir bordes y colores

```
Td,th {
    border:1px;
    border-style:solid;
    border-color:black ;
}
```

También se pueden definir las imágenes de fondo (tanto de las filas, como de las celdas, como del fondo de toda la tabla).

12 – Propiedades CSS etiqueta tabla:**[02]****Modificar márgenes**

Se pueden manipular los márgenes tanto de la tabla, como de cada uno de sus elementos.

Atributos:

```
Table {
    Margin-left:00px -00%;
    Margin-right:00px-00%;
    Margin-top:00px-00%;
    Margin-bottom: 00px-00%;
}
```

Modificar Padding

Se pueden manipular los padding tanto de la tabla, como de cada uno de sus elementos.

Atributos:

```
Table {
    padding-left:00px -00%;
    padding-right:00px-00%;
    padding-top:00px-00%;
    padding-bottom: 00px-00%;
}
```

Definir alto /ancho

A la hora de definir los tamaños, se puede hacer de cualquier sección de la tabla, o de toda en sí misma.

Al igual que con el resto de etiquetas del HTML 5, si solo definimos uno de los atributos de tamaño, el resto se asignará proporcionalmente.

Definir alto

```
table {          table td {
    height:200px;    height:50px;
}
```

Definir ancho

```
table {          table td {
    width:90px;    width:90px;
}
```

12 – Propiedades CSS etiqueta tabla:**[03]**

Por ultimo también se pueden alinear los textos o contenidos del interior de las celdas de una tabla. Para ello solo tenemos que emplear los mismos atributos que usamos en las etiquetas para alinear textos;

Alinear textos en celdas

<pre>table td { Text-align:left; }</pre>	<pre>Table td { Text-align:left; }</pre>	<pre>Table{ Text-align:center; }</pre>
<pre>Table{ Text-align:right; }</pre>	<pre>Table{ Text-align:justify; }</pre>	<pre>Table{ Text-align:end; }</pre>
	<pre>Table{ Text-align:start; }</pre>	

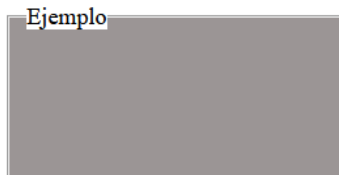
```
<table class="ejemplo">
  <thead>
<tr><th>Clave</th><th>Color</th><th>Valor</th><th>TAM</th></tr>
</thead>
<tbody>
<tr><td>A001</td><td>rojo</td><td>15</td><td>Lite</td></tr>
<tr><td>A002</td><td>azul</td><td>76</td><td>Big</td></tr>
</tbody>
<tfoot>
<tr><td>A00-</td><td>RGB</td><td>Price</td><td>Size</td></tr>
</tfoot>
</table>
```

Clave	Color	Valor	TAM
A001	rojo	15	Lite
A002	azul	76	Big
A00-	RGB	Price	Size

13 – Etiquetas de formulario:

Disponemos de múltiples opciones a la hora de crear formularios. Lo primero que se suele hacer es definir un área para estos forms.

Zona de formulario:



```
<fieldset>

  <legend>Ejemplo</legend>

</fieldset>
```

El ejemplo superior, también tiene una serie de atributos definidos en el CSS, “fieldset” define el espacio remarcado en cual interior irán emplazados los controles del formulario. “Legend” define el nombre resaltado del campo del formulario

```
fieldset {
    margin: auto;
    background-color:rgb(155, 149, 149);
    width:200px;
    height: 50%;
}

fieldset legend {
    background-color:white;
}
```

En el interior del fieldset se pueden definir una amplia variedad de controles.

Labels / inputs:

```
<fieldset>

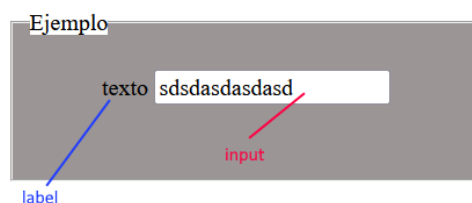
  <legend>Ejemplo</legend>

  <br>

  <label for="algun_nombre"> texto</label>

  <input type="text" id="algun_nombre" name="">

</fieldset>
```



A continuación, veremos las listas desplegables. Estas listas pueden tener un número ilimitado de ítems, así como anidar imágenes, links o href en su interior.

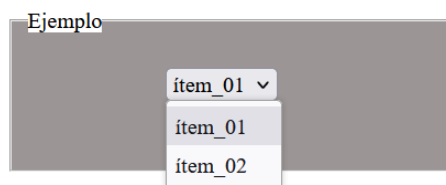
Listas:

```
<select>

  <option value="ítem_01">ítem_01 </option>

  <option value="ítem_02">ítem_02 </option>

</select>
```



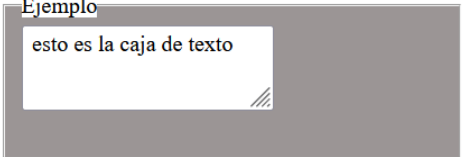
13 – Etiquetas de formulario:**[Continuación]**

También tenemos a nuestra disposición cajas de texto.

Cajas de texto:

```
<textarea rows="10" cols="1" >
    esto es la caja de texto
</textarea>
```

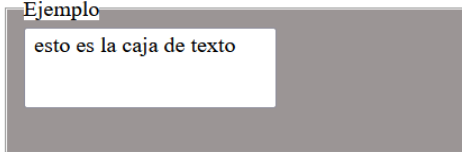
Ejemplo



Podemos eliminar el resize desde el selector de CSS.

```
textarea {
    resize:none;
}
```

Ejemplo

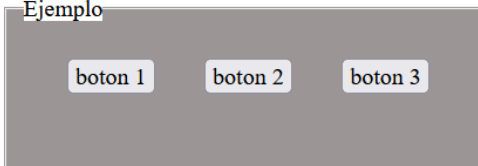


Otro elemento de los formularios son los botones. Estos poseen una gran cantidad de atributos, entre los que destacan, color, imagen de fondo, ancho y alto. Así como todos los controles de padding y margin.

Botones:

```
<button type=""> botón 1</button>
<button type=""> botón 2</button>
<button type=""> botón 3</button>
```

Ejemplo

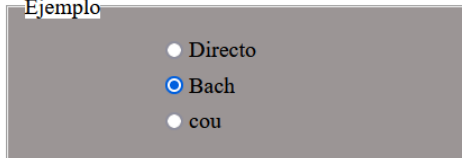


Los botones cuentan con propiedades especiales en “type”. Estos son respectivamente: button (botón), reset (resetear), submit (enviar). Estos valores de retorno, son aprovechados por scripts de java script y PHP.

Sistemas de múltiple selección:

```
<input type="radio" name="entrada" value="Directo">
<br>
<input type="radio" name="entrada" value="bach">
<br>
<input type="radio" name="entrada" value="Cou">
```

Ejemplo



En este caso se muestran tres inputs de botón circular. Al tener todos los mismos valores de “name” solo se admite una selección. Si se varía el “name”, se pueden hacer múltiples selecciones simultáneas.

13 – Etiquetas de formulario:**[Continuación]**

Mismo caso que el anterior, pero esta vez con cajas de check.

```
<input type="checkbox" name="entrada" value="Direct"> Directo
```

```
<br>
```

```
<input type="checkbox" name="entrada" value="bach"> Bach
```

```
<br>
```

```
<input type="checkbox" name="entrada" value="Cou"> cou
```

Ejemplo

☐ Directo
☒ Bach
☐ cou

Atención:

Todo lo anteriormente referenciado sobre formularios HTML5 es información “básica”. Estos formularios y controles, por si mismos no tienen más valor que el estético. Para ser efectivos, han de ser implementados con scripts en PHP / Java script. Así mismo los atributos de identificación, post y activación no han sido mencionados en este documento. Si desea emplear estos controles a de revisar la documentación completa disponible en :

<https://www.w3schools.com/html/>

14- Etiquetas**[Métodos adicionales]****[01] Editar contenido de contenedor a tiempo real**

En HTML5 podemos editar el contenido de cualquier contenedor (<div> <table> <section>...) mediante el siguiente atributo.

```
<div Contenteditable="True"> edítame </div>
```

```
<article>
```

```
<label>La parte inferior es un div editable</label>
```

La parte inferior es un div editable
 aqui estoy

```
<div contenteditable="true"> aqui estoy </div>
```

```
</article>
```

Esta característica la podemos añadir a cualquiera de las etiquetas de formulario. Inc, labels, inputs, buton...

14- Etiquetas**[Métodos adicionales]****[02]Input matriz de color a tiempo real**

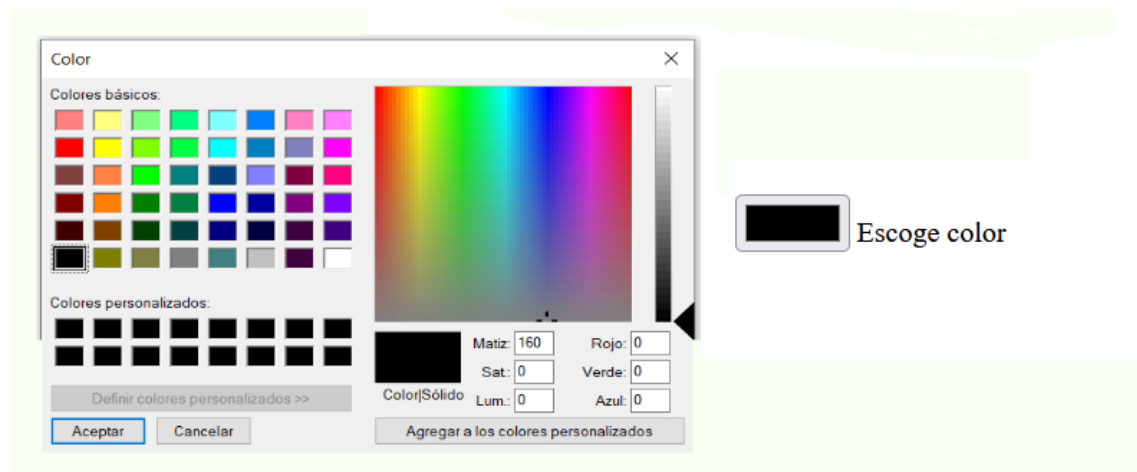
En Html5 podemos introducir inputs de matriz RGB a tiempo real (datos exportables a Java Script y PHP) Para ello simplemente tenemos que añadir el siguiente atributo a un input.

```
type="color"
```

```
<article>
```

```
<input type="color" name="rgb" value="seleccion_RGB"> Escoge color
```

```
</article>
```

**[03]Marcar textos, palabras o zonas de una página HTML**

Este método se puede aplicar tanto a zonas y textos como a etiquetas de formulario. El efecto es el mismo que el de los marcadores tradicionales.

```
<article>
```

```
<p> Ejemplo de marcar una zona de texto;
```

```
<mark>esta misma</mark>
```

```
</p>
```

```
</article>
```

Ejemplo de marcar una zona de texto; **esta misma**

Se pueden cambiar tanto el color del marcado, como el color de la fuente del texto remarcado mediante el siguiente selector de CCS.

```
Mark {
color:red;

background-color: aqua;
}
```

14- Etiquetas

[Métodos adicionales]

[04] Barra de progreso porcentual

En HTML5 podemos incluir gráficos a modo de “barras de carga” para mostrar el progreso de un determinado campo. Estas barras se pueden animar mediante Java Script.

```
<article class="barra">
```

```
<p> Barra de carga HTML5</p>
```

```
<meter min="0" max="100" value="50">esta es la barra</meter>
```

```
</article>
```

Barra de carga HTML5



Es importante destacar, que los atributos de [min], [max] y [value] no se pueden manipular desde CSS, puesto que no forman parte de ningún selector. Su definición y manipulación se tiene que efectuar desde el HTML.

Esta barra posee algunos selectores en CSS, siendo el mas útil el que define su tamaño en pantalla.

```
meter {  
    width: 200px;  
}
```

Tenemos otro método para mostrar una “barra” de progreso, en este caso trataríamos la etiqueta [progress] su funcionamiento es exactamente igual al de [meter]. La única diferencia es que si introducimos un texto entre su etiqueta de apertura y la de cierre este, aparecerá (en algunos navegadores) en el interior de la barra.

```
<article class="barra">
```

```
<p> Barra de progreso HTML5</p>
```

```
<progress min="0" max="100" value="50">50%</progress>
```

```
</article>
```

```
</section>
```

Barra de progreso HTML5



14- Etiquetas**[Métodos adicionales]****[05] Creación de mensajes emergentes desde HTML5**

Este método es una novedad del HTML5, a diferencia que las anteriores versiones ya no son necesarios el uso de scripts para mostrar mensajes o banners emergentes. Estos diálogos, pueden tener botón de cierre, activación o ambos. Así mismo se pueden introducir más etiquetas y datos en ellos.

```
<article>
```

```
<button onclick="ejemplo.showModal()">abrir</button>
```

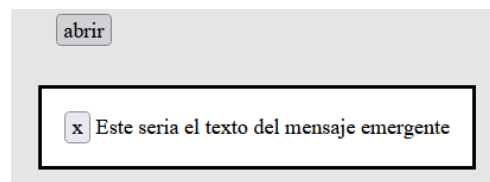
```
<dialog id="ejemplo">
```

```
<button onclick="ejemplo.close()">x</button>
```

```
Este seria el texto del mensaje emergente
```

```
</dialog>
```

```
</article>
```



En este código estaríamos empleando un botón para “abrir” el dialogo, y otro botón (en el interior del banner) para cerrarlo. Estos botones se pueden editar tanto en contenido, apariencia como en ubicación mediante su respectivo selector en CSS.

Es importante tener en cuenta que el propio “dialog” se puede manipular desde su selector CSS, pudiendo modificar tanto su tamaño como apariencia. Su z-index siempre es superior al del elemento más emergente de la página, lo cual lo sitúa sin excepción por encima de los contenidos.

Atención;

Esta es una etiqueta con Case sensitive, lo cual determina que si no escribe exactamente igual a lo referenciado no funciona.

ejemplo,showModal() [no es lo mismo que] ejemplo.showmodal()

Ejemplo de selector de dialog:

```
dialog {
```

```
width: 50%;
```

```
height: 100%;
```

```
background-color: brown;
```

```
}
```

En este ejemplo, estaríamos definiendo el ancho del dialog al 50% del tamaño de la página, así como el 100% de la altura de la misma.

Por último, asignaríamos un background opaco de color marrón.

15- Selectores**CSS**

Los selectores son los elementos que se emplean a la hora de editar los estilos en CSS, disponen de varios métodos de selección.

Importante:

Los estilos situados en el interior del selector SIEMPRE se finalizan con un [;] de lo contrario el selector no se aplica, y en la mayoría de las ocasiones, se producen errores graves en el renderizado del documento HTML.

Selector1,selector2{}	Los estilos se aplican a todas las etiquetas-selectores seleccionados.
Selector1*{}	Los estilos se aplican a todos los hijos del selector seleccionado.
Selector1 selector2{}	Los estilos se aplican a todas las etiquetas del selector 2 dentro del selector 1.
Selector1>selector2{}	Se cambian las propiedades de los elementos seleccionados por selector 2 que son hijos directos del selector1.
*{}	Selector universal. Se cambian/definen las propiedades de todos los elementos del documento HTML5.

Selectores de uso común

Existe infinidad de selectores y sub-selectores, cada etiqueta o método posee una pequeña lista. Los expuestos a continuación son una brevísima selección de los que más se emplean.

	Color texto	Imagen fondo
Selector { Color:red; background-color: aqua; }	Color:red; Background-color:aqua;	Define el color del texto. Define el color de fondo
Selector { margin-left: 350px; height:200px; background-image: url(../img/log_00.png); }	Margin-left:350px; Height:200px; Background-image: url(../img/log_00.png);	Define el margen izq. Define la altura. Define la imagen de fondo del elemento de donde procede el selector.

Selectores de uso común**[Continuación]**

Con cualquier tipo de selector se puede definir tanto el ancho como el alto de su etiqueta o elemento referencial. Así mismo también se pueden emplear para manipular los padding.

Tamaño**Márgenes**

```
selector{
  width:00px;
  height:00px;
} | en px o %

selector{
  margin:00px;
  margin-left:00px;
  margin-top:00px;
  margin-bottom:00px;
  margin-right:00px;
} | en px o %
```

Importante:

En ambos casos se pueden definir los valores en Px o %.

Px: Valor fijo, no se puede modificar.

%: Adaptable a la superficie de renderizado, el porcentaje se calcula en base al width del contenedor padre.

Todos los elementos de HTML5 son contenedores, o se comportan como tal. Por ello mediante los siguientes selectores, se pueden definir sus bordes y las características de los mismos.

Color bordes**Bordes**

```
selector{
  border-color: black;
  border-width:1px;
  border-style:solid;
}

selector{
  border-left: 1px;
  border-right: 1px;
  border-top:1px;
  border-bottom: 1px;
}
```

Importante:

En ambos casos lo correcto es emplear los valores en Px.

Px: Valor fijo, no se puede modificar.

En el caso de la definición de bordes (lef,right,top,bottom) si queremos no emplear uno de estos bordes, solo tenemos que definirlo a 0px.

Una de las características más empleadas en HTML5 son la transparencia y el bordeado de las esquinas.

```
dialog {
  width: 50%;
  background-color: brown;
  border-radius: 15px;
  opacity: 0.5;
}
```

Redondeado**Transparencia**

Opacity: Define la transparencia del elemento al que hace referencia el selector.

Border-radius: Se expresa en Px, redondea las esquinas.

16- Uso de fuentes

En HTML5 se pueden emplear diferentes fuentes de texto; tanto cargándolas desde un repositorio en internet (fonts.google.com) como mediante una carga desde el directorio que hospeda la propia webside. En el curso que nos ocupa únicamente haremos uso de la carga remota, empleando de este modo el procedimiento de Google fonts.

Atención:

Dada la laboriosidad y extensión del procedimiento, en el presente manual, no detallaremos la búsqueda, localización y adaptación de los scripts en el propio Google fonts. Para más información, consulte los meets de clase AW o algún video tutorial.

Fuentes recomendadas para títulos

Noto sans / Lato / Open

Titulos: Noto sans

link: <https://fonts.google.com/noto/specimen/Noto+Sans?query=Noto+sans&subset=latin>

regular 400

En css:

```
selector{  
  font-family: 'Noto Sans', sans-serif;  
}
```

En head:

```
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
<link href="https://fonts.googleapis.com/css2?family=Noto+Sans&display=swap"  
rel="stylesheet">
```

16- Uso de fuentes

[Continuación]

Fuentes recomendadas para títulos

Títulos: lato

<https://fonts.google.com/specimen/Lato?query=lato&subset=latin>

light 300

En css:

```
font-family: 'Lato', sans-serif;
```

En head:

```
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
<link href="https://fonts.googleapis.com/css2?family=Lato:wght@300&display=swap" rel="stylesheet">
```

16- Uso de fuentes

[Continuación]

Títulos : open

[url:https://fonts.google.com/specimen/Open+Sans?query=open&subset=latin](https://fonts.google.com/specimen/Open+Sans?query=open&subset=latin)

medium 500

En css:

```
font-family: 'Open Sans', sans-serif;
```

En head:

```
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
<link  
href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@500&display=swap"  
rel="stylesheet">
```

16- Uso de fuentes**[Continuación]****Fuentes recomendadas para textos****Textos:** Roboto [tamaño pequeña]<https://fonts.google.com/specimen/Roboto+Mono?query=roboto>**En css:**

```
font-family: 'Roboto Mono', monospace;
```

En head:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Roboto+Mono:wght@100&display=swap"
rel="stylesheet">
```

Textos: Roboto [tamaño pequeña] regular 400 / 16**En css:**

```
font-family: 'Roboto', sans-serif;
```

En head:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"
rel="stylesheet">
```

17- Seudo-selectores**:hover****:focus**

Existen diferentes tipos de selectores en CSS, el siguiente grupo hace referencia a estados o interacciones.

```
.ejemplo tbody tr:hover {  
background-color: aqua;  
height:90px;  
}
```

Clave	Color	Valor	TAM
A001	rojo	15	Lite
A002	azul	76	Big
A00-	RGB	Price	Size

- .ejemplo** Clase que referencia la tabla (definida previamente en el HTML)
- Tbody** zona concreta del cuerpo de la tabla (definida previamente en HTML)
- Tr:hover** Este seudo-selector hace referencia a todos los elementos <tr> de la zona anteriormente citada. [hover] es el estado de “pasar el ratón por encima de ella”.

El seudo-selector superior cambia el color y el alto de las celdas de la fila (tr) del cuerpo de la tabla (tbody) cuando el ratón pasa por encima de cualquier segmento de las filas de esa zona.

```
.ejemplo tbody tr:focus {  
background-color:brown;  
height:90px;  
}
```

Este elemento funciona igual que el anterior, pero en este caso, en lugar de activarse al pasar el ratón por encima de él, se activa al ser el “foco” de atención. Esto se suele conseguir mediante la tabulación.

Atención:

Es importante destacar que estos seudo selectores se aplican a cualquier etiqueta o atributo, no “solo” a <tr> y a tablas. Para emplearlo con otra etiqueta solo hay que cambiar el identificador del elemento.

Ejemplo:

```
Button:hover {  
Width:150px;  
Color:red;  
}
```

En este caso, al pasar el ratón por encima del botón, su ancho cambiaria a 150px y el color de su texto pasaría a ser rojo.

17- Seudo-selectores**:first-child**

Estos seudo-selectores, aplican los formatos de diferente modo.

.ejemplo tr:first-child {

background-color: aqua;

height:40px;

}

El seudo-selector de :first-child, cuenta los elementos de 2 en 2, aplicando los formatos solo al 1 elemento de cada 2.

Esto quiere decir que si el número de elementos del selector es impar, el resultado puede no ser el esperado.

<tr>	Clave	Color	Valor	TAM	<tr> 1
<tr>	Clave	Color	Valor	TAM	<tr> 2
<tr>	A001	rojo	15	Lite	<tr> 1
<tr>	A002	azul	76	Big	<tr> 2
<tr>	A00-	RGB	Price	Size	<tr> 1
<tr>	A00-	RGB	Price	Size	<tr> 2

Mediante estos seudo-selectores también podemos seleccionar el primer tipo específico (el que designemos) en una etiqueta concreta.

:first-of-type

.ejemplo td:first-of-type {

background-color: aqua;

height:40px;

}

En este caso, :first-of-type aplica el formato definido en el primer elemento de la clase referenciada de cada una de las filas.

Si se aplica fuera de una tabla, su funcionamiento es igual, salvo que en lugar de "filas" cuenta las líneas de código.

<tr>	<th>	Clave	Color	Valor	TAM	<tr>	<th>		
<tr>	<td>	Clave	<td>	Color	<td>	Valor	<td>	TAM	</tr>
<tr>	<td>	A001	<td>	rojo	<td>	15	<td>	Lite	</tr>
<tr>	<td>	A002	<td>	azul	<td>	76	<td>	Big	</tr>
<tr>	<td>	A00-	<td>	RGB	<td>	Price	<td>	Size	</tr>
<tr>	<td>	A00-	<td>	RGB	<td>	Price	<td>	Size	</tr>

17- Seudo-selectores**:nth-child(n)**

Este es un selector dinámico, ello quiere decir que emplea una variable para “moverse” en los campos de la tabla o elemento donde este referenciado.

Su sintaxis es la siguiente:

elemento:nth-child(n+a)

n Es la posición 0 de la tabla o elemento referenciado.

a Es la posición que se suma al 0 y la que define el punto a partir del cual se aplican los formatos.

Ejemplo:

```
.ejemplo td:nth-child(n+2) {
background-color: aqua;
height:40px;
}
```

En este caso hemos indicado (n+2) como posición de inicio, por lo que el formato se aplica a partir del segundo <td> de cada fila.

Clave	Color	Valor	TAM
Clave	Color	Valor	TAM
A001	rojo	15	Lite
A002	azul	76	Big
A00-	RGB	Price	Size
A00-	RGB	Price	Size
1 <td>	2 <td>	3 <td>	4 <td>

Ejemplo:

```
.ejemplo td:nth-child(n+3) {
background-color: aqua;
height:40px;
}
```

En este caso hemos indicado (n+3) como posición de inicio, por lo que el formato se aplica a partir del tercer <td> de cada fila.

Clave	Color	Valor	TAM
Clave	Color	Valor	TAM
A001	rojo	15	Lite
A002	azul	76	Big
A00-	RGB	Price	Size
A00-	RGB	Price	Size
1 <td>	2 <td>	3 <td>	4 <td>

17- Seudo-selectores

:nth-child(n)**even****odd**

Podemos emplear el seudo-selector `:nth-child(n)` para seleccionar solo los elementos pares o impares.

Sintaxis:**:nth-child(even)**

Seleccionaremos únicamente los elementos pares de la clase que tengamos en el selector.

:nth-child(odd)

Seleccionaremos únicamente los elementos impares de la clase que tengamos en el selector.

Ejemplo:

```
.ejemplo tr:nth-child(even) {
background-color: aqua;
height:40px;
}
```

Al emplear [even] el seudo selector, solo aplica el formato indicado en los elementos pares.

<tr> 1	Clave	Color	Valor	TAM
<tr> 2	Clave	Color	Valor	TAM
<tr>3	A001	rojo	15	Lite
<tr>4	A002	azul	76	Big
<tr>5	A00-	RGB	Price	Size
<tr>6	A00-	RGB	Price	Size

Ejemplo:

```
.ejemplo tr:nth-child(odd) {
background-color: aqua;
height:40px;
}
```

Al emplear [odd] el seudo selector, solo aplica el formato indicado en los elementos impares.

<tr> 1	Clave	Color	Valor	TAM
<tr> 2	Clave	Color	Valor	TAM
<tr>3	A001	rojo	15	Lite
<tr>4	A002	azul	76	Big
<tr>5	A00-	RGB	Price	Size
<tr>6	A00-	RGB	Price	Size

17- Seudo-selectores**:last-child****:last-of-type**

También podemos seleccionar mediante los seudo-selectores referenciada.

```
.ejemplo td:last-child {
background-color: aqua;
height:40px;
}
```

El seudo selector `:last-child` en este caso, busca cuales son los `<td>` situados al final de cada fila, aplicándole el formato.

El seudo-selector `:last-of-type` funciona exactamente igual.

Clave	Color	Valor	TAM
Clave	Color	Valor	TAM
A001	rojo	15	Lite
A002	azul	76	Big
A00-	RGB	Price	Size
A00-	RGB	Price	Size

1 2 3 4
<td> <td> <td> <td>

18- Seudo-elementos**::first-line****::first-letter**

Mediante los seudo elementos podemos definir los formatos de bloques de texto o similares.

Ejemplo:

```
.ejemplo_AA p::first-line {
font-weight: bold;
}
```

Lorem ipsum dolor sit amet,
consectetur adipisicing elit.
Sit a commodi incidunt
asperiores totam quo.

En este ejemplo hemos creado un párrafo de texto mediante `lorem15`, dividiéndolo con `<br>` creando 4 líneas de texto.

Para aplicar el formato de negrita a la primera línea, simplemente asignamos el seudo-elemento a la etiqueta `[p]` de la clase `[.ejemplo_AA]`.

Este sub-elemento solo funciona si esta aplicado a un contenedor de texto.

Ejemplo:

```
.ejemplo_AA p::first-letter {
font-weight: bold;
}
```

Lorem ipsum dolor sit amet,
consectetur adipisicing elit.
Sit a commodi incidunt
asperiores totam quo.

En este caso al aplicar el seudo-elemento, este busca la primera letra del párrafo de texto y le aplica el formato definido (negrita).

18- Seudo-elementos

::after

::before

::selection

Podemos definir textos, antes y después de un párrafo, línea o contenedor de texto. Estos seudo-elementos siempre mostraran los textos independientemente de la cantidad de veces que se repitan o la parte del documento HTML donde estén.

Ejemplo:

```
.ejemplo_AA p::after {
  font-weight: bold;
  content: "Esto es lo que pondríamos 'después'";
}
```

Lorem ipsum dolor sit amet,
consectetur adipisicing elit.
Sit a commodi incidunt
asperiores totam quo. **Esto es lo que
pondríamos 'después'**

En este caso al aplicar el seudo-elemento, muestra el texto que previamente hemos introducido en el campo “content” referenciando a los párrafos <p> de la clase .ejemplo_AA al final del párrafo.

Ejemplo:

```
.ejemplo_AA p::before {
  font-weight: bold;
  content: "Esto es lo que pondríamos 'antes' ";
}
```

Esto es lo que pondríamos 'antes' Lorem
ipsum dolor sit amet,
consectetur adipisicing elit.
Sit a commodi incidunt
asperiores totam quo.

En este caso al aplicar el seudo-elemento, muestra el texto que previamente hemos introducido en el campo “content” referenciando a los párrafos <p> de la clase .ejemplo_AA al comienzo del párrafo.

Ejemplo:

```
* ::selection {
  color: red;
  background: yellow;
}
```

Esto es lo que pondríamos 'antes' Lorem
ipsum **dolor sit amet,**
consectetur adipisicing elit.
Sit a commodi incidunt
asperiores totam quo.

La seudo clase ::selection nos permite hacer visible y diferente la selección de fragmentos de texto o contenidos en nuestra web.

En este caso, el texto se vuelve de color “rojo” y el fondo “amarillo”.

19- CSS Flex

En HTML 5 disponemos de diferentes métodos de trabajo a la hora de maquetar contenidos web. Uno de los más sencillos y a la par potentes es FLEX.

19-1 ¿En qué consiste FLEX?

La idea principal de flex, es la de facilitar la maquetación de la webside por medio de cajas, que a su vez están dentro de otras cajas creadas en las etiquetas semánticas.

19-2 ¿Qué puede ser un contenedor flex?

Cualquier etiqueta semántica, o que por sí ya se emplee como contenedor.

Ejemplos:

[HTML]

```
<main class="contenedor">    <section class="contenedor">    <div class="contenedor">
</main>                    </section>                    </div>
```

19-3 Declaración de contenedor Flex:

El paso imprescindible para poder hacer uso de los métodos de flex es configurar el contenedor como flex.

Ejemplo:

[CCS]

```
.contenedor {
display: flex;
}
```

Para que una etiqueta o semántica, sea Flex solo hay que asignarle la propiedad "display=flex" en su selector.

El nombre del selector es completamente arbitrario, no es obligatorio que sea "contenedor".

19-4 Declaración de dirección de los elementos Flex:

[row]

Hay tres tipos de configuraciones de direccionamiento de los elementos Flex, el primero de ellos es el que define la dirección por defecto de los elementos.

```
.contenedor {
Flex-direction: row;
}
```

Esta es la configuración por defecto de flex, los elementos se organizan horizontalmente de izquierda a derecha.

19- CSS Flex**[Continuación]**

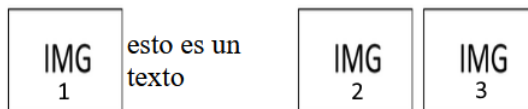
A continuación, vamos a declarar un contenedor Flex en un artículo que forma parte de una sección.

Ejemplo:

```
<article class="contenedor_ejemplo">

<p> esto es un texto</p>


</article>
```



Se puede apreciar como los elementos se han alineado por defecto en modo [row]

En CSS:

```
.contenedor_ejemplo img{
width: 25%;
}
.contenedor_ejemplo {
display: flex;
}
```

En el documento HTML hemos creado el contenedor en un artículo, y posteriormente le hemos introducido tres imágenes y un pequeño texto.

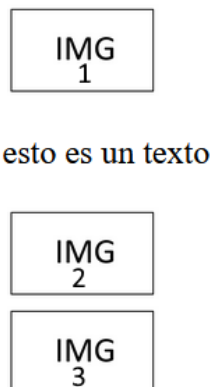
En el CCS hemos definido el display flex en el selector del contenedor y también hemos reducido el tamaño de las IMG para que pudieran gestionarse correctamente.

19-5 Declaración de dirección de los elementos Flex:**[column]**

La segunda de las alineaciones básicas de las que disponemos es la [column] en esta en lugar de ordenarse los elementos de izquierda a derecha en horizontal, se ordenan de arriba hacia abajo en vertical.

En CSS:

```
.contenedor_ejemplo {
display: flex;
flex-direction: column;
}
```



19- CSS Flex**[Continuación]****19-6 Declaración de dirección de los elementos Flex:****[reverses]**

Los elementos siguen un orden concreto a razón del tipo de alineación que este definida, este orden puede ser invertido mediante los “reverse”.

flex-direction: row-reverse; En lugar de ordenarse de izquierda a derecha en horizontal, pasan a ordenarse de derecha a izquierda en horizontal.

flex-direction: Columnm-reverse; En lugar de ordenarse de arriba hacia abajo, en vertical, pasan a ordenarse de abajo hacia arriba en vertical.

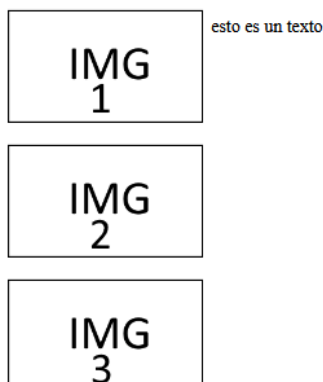
19-7 Ajuste de los elementos Flex:

Flex dispone de un método para definir el comportamiento de los elementos que se desborden del contenedor.

flex-wrap: nowrap; Este es el método por defecto que emplea Flex ante los elementos que se desbordan. Fuerza a mantener todos los elementos en la misma línea.



flex-wrap: wrap; Los elementos que se desbordan pasan a la línea inferior, en múltiples líneas si así lo requieren, de arriba hacia abajo.



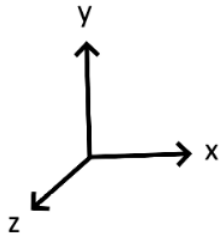
Flex-wrap , también dispone de un método reverse.

flex-wrap: wrap-reverse;

De este modo al desbordarse se ordenan los elementos de abajo hacia arriba.

19- CSS Flex**[Continuación]****19-8 Alineación horizontal de los elementos Flex:**

Los elementos Flex cuentan con dos ejes de alineación, en correspondencia a los ejes cartesianos [X] e [Y]

**Equivalencias**

Eje X: Justify-content; flex-start, flex-end, center, space between, Space-around, space-evenly.

Eje Y: Align-items ; Flex-start, flex-end, center, stretch, baseline.

Eje Z: z-index.

19-9 Alineación horizontal**[Justify-content]**

Esta propiedad de Flex, nos permite ajustar los contenidos en el eje horizontal (x), poniendo a nuestra disposición una serie de ajustes.

Importante

Todas las propiedades de alineación de FLEX solo son efectivas (y visualizables en el renderizado de la webside) si el tamaño del contenedor padre, es lo suficientemente grande para permitir la ordenación. CSS ignorara las configuraciones de alineación, si el tamaño del id contenedor es demasiado pequeño o “justo”. Esto se puede comprobar haciendo sólidos y visibles los bordes de la clase contenedora.

Justify-content: flex-start;

Mediante esta configuración los elementos se ordenarían desde el punto de inicio de FLEX. Por defecto, la izquierda de la pantalla, si estamos en reverse, la derecha.

**Justify-content: flex-end;**

Mediante esta configuración los elementos se ordeñarían desde el punto de final de FLEX. Por defecto la parte derecha de la pantalla. Si estamos en reverse; la izquierda.



19-9 Alineación horizontal

[Justify-content]**Justify-content:center;**

Mediante esta configuración los elementos se agruparían en el centro del contenedor FLEX.



Además de las alineaciones que agrupan los componentes del FLEX, también disponemos de alineaciones con espaciados.

Justifi-content:space-between;

Esta configuración reparte los elementos, dejando el mismo espaciado entre los centrales.

**Justifi-content:space-around;**

Esta configuración reparte los elementos dejando un espacio menor entre los elementos centrales, y un mismo espacio en el de inicio y final.

**Justifi-content:space-evenly;**

Esta configuración iguala los márgenes de inicio y final, repartiendo el resto de los elementos en el centro.



19-10 Alineación vertical**[Align-items]**

Esta propiedad de Flex, nos permite ajustar los contenidos en el eje vertical (y), poniendo a nuestra disposición una serie de ajustes.

Align-items:flex-start;

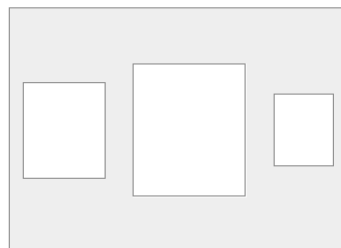
Esta configuración ajusta los elementos al borde superior del contenedor padre.

**Align-items:flex-end;**

Esta configuración ajusta los elementos al borde inferior del contenedor padre.

**Align-items:center;**

Esta configuración ajusta los elementos al centro del contenedor padre.

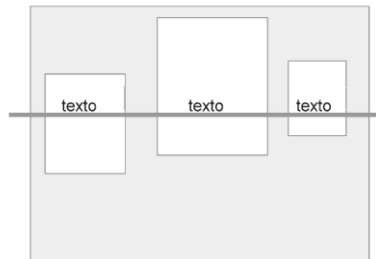
**Align-items:stretch;**

Esta configuración “estira” los elementos deformando su eje y, manteniéndolos en el centro del contenedor.



19-10 Alineación vertical**[Align-items]****Align-items:baseline;**

Esta configuración centra las imágenes sobre una línea imaginaria donde se ubican los textos.

**19-10 Alineación vertical-wrap****[Align-items]**

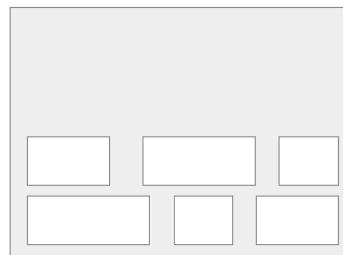
Si tenemos activado el wrap (flex-wrap: wrap;) el comportamiento de las alineaciones varía, dándose unos resultados totalmente diferentes a los comunes. Una vez más, si el contenedor no tiene un tamaño suficientemente grande tanto de vertical como de horizontal, estas alineaciones no funcionarían.

Align-items:flex-start;

Esta configuración ajusta los elementos al borde superior del contenedor padre.

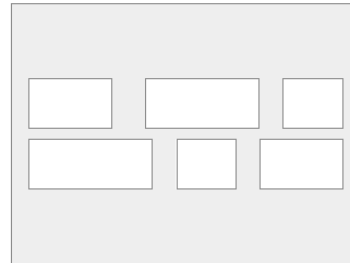
**Align-items:flex-end;**

Esta configuración ajusta los elementos al borde inferior del contenedor padre.



19-10 Alineación vertical-wrap**[Align-items]****Align-items:center;**

Esta configuración ajusta los elementos al centro del contenedor padre.

**Align-items:stretch;**

Esta configuración “estira” los elementos deformando su eje y, manteniéndolos en el centro del contenedor.

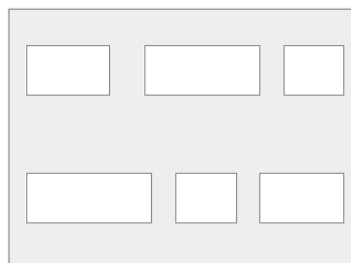
**19-11 Alineación horizontal-wrap****[Justifi-content]****Justifi-content:space-between;**

Esta configuración reparte los elementos, dejando el mismo espaciado entre los centrales.



19-11 Alineación horizontal-wrap**[Justifi-content]****Justifi-content:space-around;**

Esta configuración reparte los elementos dejando un espacio menor entre los elementos centrales, y un mismo espacio en el de inicio y final.

**19-12 Orden de aparición de los elementos FLEX**

El orden de colocación de los elementos FLEX viene por defecto, asignado por su aparición en el código. Este orden, puede ser modificado a voluntad, asignando previamente un ID o Clase diferente a cada uno de los elementos del flex.

Ejemplo:

```
<article class="contenedor_ejemplo">

<p id="segundo"> esto es un texto</p>


</article>
```

Class="contenedor_ejemplo" [Contenedor]**Id="primero" [Elemento FLEX 01]****Id="segundo" [Elemento FLEX 02]****Id="tercero" [Elemento FLEX 03]****Id="cuarto" [Elemento FLEX 04]**

Una vez hemos asignado un id o una clase a cada uno de los elementos, ya podemos alterar su orden de aparición empleando su respectivo selector en el CSS.

En CSS:

```
#primero {
  order:2;
}
#segundo {
  order:1;
}
```

esto es un texto



En este caso la imagen 00.png pasaría a estar en el lugar de la que ocupaba el texto, y el texto al lugar que ocupaba la imagen 00.png

19-13 Deformación de los elementos FLEX

En algunas ocasiones, necesitaremos “rellenar” espacios en los contenedores FLEX. En principio esto se puede configurar mediante los atributos de `width` y `height` en CSS de modo manual. En FLEX disponemos de una serie de métodos que facilitan esta labor.

Atención:

Para poder emplear estos métodos de modo eficiente es importante complementarlos de dos atributos nuevos CSS (que no entran en el curso de AW) :

max-height: 150px;	Mediante este parámetro, definimos la altura máxima que puede alcanzar el elemento al que hace referencia el selector. Se puede definir en PX o %.
max-width: 500px	Mediante este parámetro, definimos el ancho máximo que puede alcanzar el elemento al que hace referencia el selector. Se puede definir en PX o %.

El parámetro que tenemos que definir en el selector es `[flex-grow]`, este se tiene que activar en cada uno de los elementos que deseemos se deformen para cubrir los espacios vacíos. El valor para la activación es `[1]` cualquier otro número mayor no variara su efecto.

En CSS

#primero {	#cuarto {
flex-grow: 1;	flex-grow: 1;
max-height: 150px;	max-height: 150px;
max-width: 500px;	max-width: 500px;
}	}

Para poder aplicar este parámetro es necesario haber declarado previamente una clase o un id de los elementos que requiramos.

Se declara a `[1]` y para que sea más eficiente, delimitamos los tamaños máximos.

Antes del Flex-grow:**Con el Flex-grow:**

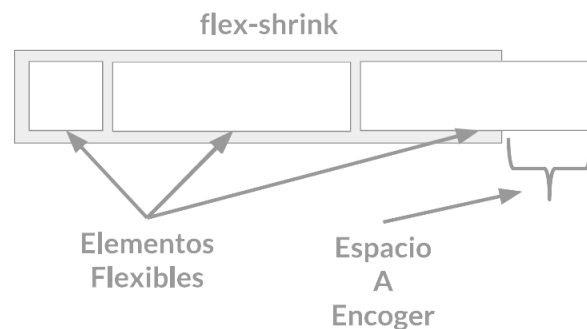
19-13 Deformación de los elementos FLEX

En algunas ocasiones los elementos FLEX desbordan la clase contenedor. Disponemos de una opción para poder reducir el ancho del objeto desbordante de modo automático.

Flex-shrink:1;

Este es el factor que activa la contracción de un elemento flexible cuando sobrepasa el tamaño de su contenedor

Se declara a [1] y tiene que hacerse en todos los elementos que necesitamos, de manera individual.

**Flex-basis: npx**

Tamaño que tiene que alcanzar un elemento (deformándose) antes de que se distribuya por el espacio restante (negativo o positivo). (Auto por defecto)

20 – Alineación individual en grupos de componentes FLEX

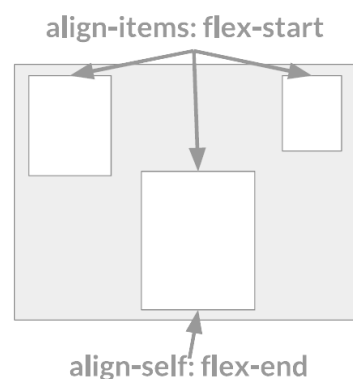
Las alineaciones FLEX suelen englobar a todos los elementos incluidos en la clase contenedor, aun así, disponemos de un atributo que nos permite asignar alineaciones a elementos aislados.

align-self: flex-end;

Mediante este parámetro podemos aplicar alineaciones a un elemento individual.

En CSS:

```
#segundo {
  order:1;
  align-self: flex-end;
}
```

Disponibles:**Flex-end;****Flex-start;****Center;****Stretch;****Baseline:****[Revisar secciones anteriores]****[061122-17:44]**

21 – Métodos de posicionamiento de elementos HTML

Los elementos que manejamos en html, heredan por defecto el tamaño de su etiqueta padre. Su posición siempre está en relación al propio documento HTML.

Ejemplo:

Si definimos un main con un width:100%, y en su interior un <div> este elemento ocupara el100% del ancho de la pantalla, siendo su ancho (height) el correspondiente al que posea su contenido.

Su posición en pantalla estaría directamente relacionada con la posición que ocupa (orden de aparición) en el documento html.

Importante:

Cuando manipulamos el width o el height de un elemento anidado (en el interior de otra etiqueta) estamos manipulando las cifras en proporción a las del elemento padre. Es importante tener esto en cuenta.

21.1 Position:absolute;

EL position:absolute, se define en el interior de la clase que defina el objeto o contenedor (en el css).

```
.imagen_noticia {  
Position:absolute;  
Left:150px;  
}
```

Donde:

Position = es el lugar de la pantalla donde está el elemento "físicamente".

Left = mover el objeto 150px hacia la izquierda.

Al declarar un elemento "**position:absolute**" este mismo elemento pierde la posición "heredada". Por lo cual, si queremos manipularlo, tendremos que reubicarlo manualmente.

Position:absolute;



caja_01 [bottom:100px;]



Así mismo, al emplear este atributo, el elemento pierde el espacio que le correspondía por posición en el html, esto quiere decir que podemos moverlo mediante los atributos:

right, left, top, bottom

El punto de referencia que empleara el elemento sigue esta jerarquía:

- 1- El elemento en el que este anidado (definiendo su Max de ancho u alto).
- 2- Su posición en el html (por defecto, adopta los Max de la semántica en la que este).
- 3- La posición en la pantalla.

21 – Métodos de posicionamiento de elementos HTML**[Continuación]****21.2 Position:relative;**

EL position:relative, se define en el interior de la clase que defina el objeto o contenedor (en el css).

```
.imagen_noticia {
  Position:relative;
  Left:150px;
}
```

Donde:

Position = es el lugar de la pantalla donde está el elemento "físicamente".

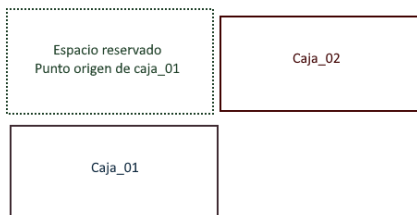
Left = mover el objeto 150px hacia la izquierda.

Al declarar un elemento "**position:relative**" este mismo elemento mantiene su posición heredada, aunque permite reubicarlo manualmente. En un principio, no notaremos ninguna diferencia.

Position:absolute;



caja_01 [bottom:100px;]



El método para moverlo será el mismo que el empleado en **[position:absolute]**

right, left, top, bottom

El punto de referencia que empleara el elemento sigue esta jerarquía:

- 1- El lugar donde estaba ubicado en principio.
- 2- Su posición en el html (por defecto, adopta los Max de la semántica en la que este).

La principal diferencia de este método frente al anterior, es que los movimientos que definamos siempre serán en referencia al lugar que ocupaba el elemento inicialmente.

Así mismo ese lugar, permanecerá "reservado" en el navegador, de modo que salvo que lo forcemos, ningún otro elemento podrá ocupar el lugar donde originalmente estaba el elemento al que asignamos el "position:relative".

21.3 Position:fixed;

El elemento que definamos como "fixed" perderá su posición de referencia html, y al igual que "position:relative" podremos moverlo libremente por todo el html, siguiendo como referencia, los máximos definidos en la semántica en la que este. A diferencia de los anteriores métodos, el "position:fixed" fija el elemento a la pantalla (igual que sticky).

21 – Métodos de posicionamiento de elementos HTML

[Continuación]

21.4 position:sticky;

A la hora de posicionar un elemento en pantalla podemos tener la necesidad de fijarlo en un punto concreto. Para ello empleamos el método “Sticky”.

<pre>.sticky { position: -webkit-sticky; /* Safari */ position: sticky; top: 0; }</pre>	El nombre de clase que invoque el elemento sticky es indiferente. Lo importante es el contenido aquí mostrado.
---	--

Es importante recordar que el método que se empleaba antes del HTML5 (`position: fixed`) actualmente está en desuso y fuera de las recomendaciones. Si se requiere fijar un elemento a la pantalla, este el método correcto que hay que emplear.

Anexo_HTML//////////[ini]

00-Eliminar efectos en href:

```
/*En tu hoja de estilos */
a:link, a:visited, a:active {
    text-decoration:none;
}
```

01-Dejar fijo el background de una website:

```
body {
background-
image:url(../img/boxed_bg.jpg);
background-attachment: fixed;
}
```

02-Ocultar/mostrar elemento html:

```
.class_elemento {
display:none; [display:yes;]
}
```

03- Modificar transparencia de un elemento:

```
.class_elemento{
Opacity:1.5;
}
```

Anexo HTML////////////////////////////////////[end]

22-Grid

Grid es un sistema de maquetación que nos permite maquetar el contenido de una website mediante el uso de rejillas.

Funcionamiento:

Grid es un modo “display” que hay que asignar a una semántica o contenedor. Se recomienda emplear (siempre que se pueda) en etiquetas semánticas prioritariamente. Su proceso de definición es exactamente igual que el Flex.

Importante:

A la hora de definir el elemento “contenedor” no hay que olvidar que no se pueden atribuir atributos “display” a las etiquetas semánticas. Si se procede de este modo, el navegador ignorara el atributo.

22.1-Definición del contenedor Grid (inicial):

En html	En CSS
(1) <div class="contenedor"> </div>	.contenedor { Display:grid; }
(2) <main class="contenedor"> </main>	main { width: 100%; margin: auto; margin-top: 50px; } .contenedor { Display:grid; }

- Si se define en un contenedor (1) el atributo “display” se puede definir inmediatamente en CSS, y trabajar con el sin problemas.
- Si se define en una semántica (2) se tienen que tratar las propiedades de la semántica y el atributo “display” en segmentos de CSS diferentes. Empleando el selector para atribuir el display grid.

22.2 Propiedades del Grid

[Definir columnas y filas]

Grid posee múltiples propiedades, siendo dos de ellas imprescindibles para poder emplearlo. Estas propiedades son:

```
grid-template-columns: 3,60%,20%,20%;
```

```
grid-template-rows: repeat(6,75px);
```

Grid-template-columns:

Esta propiedad define el número de columnas y el ancho que van a tener cada una de ellas. Hay que definir un valor de ancho (porcentaje preferiblemente para favorecer el correcto funcionamiento del adaptative Web) esto se puede definir en porcentaje o pixeles.

Se recomienda “dibujar” el grid en un papel, antes que definir estas propiedades en el CSS.

```
grid-template-columns: 3,60%,20%,20%;
```

En este ejemplo estaríamos definiendo 3 columnas, siendo la primera de 60% de ancho, y tanto la segunda como la tercera de 20%.

Importante:

Los porcentajes siempre son en “proporción” al espacio que ocupan los elementos (columna o fila) dentro de la clase contenedora. El porcentaje “real” de espacio en el visor siempre se hereda de la semántica padre. Ejemplo; si se define la clase “contenedora” en una semántica “main” el width de “main” sería el valor 100% sobre el que el contenedor repartiría los elementos del grid. El width padre, siempre hace referencia al porcentaje de uso del visor/pantalla.

Grid-template-rows:

Esta propiedad define el número de filas y el ancho que van a tener cada una de ellas. Hay que definir un valor para su anchura (preferiblemente en pixeles [px] para favorecer el correcto funcionamiento del adaptative Web) esto se puede definir en porcentaje o pixeles.

Puesto que los valores de grosor de las filas son repetitivos (salvo algunas excepciones) se recomienda emplear el método “repeat”.

```
grid-template-rows: repeat(6,75px);
```

En este caso se están definiendo 6 filas de 75px cada una.

22.2 Propiedades del Grid**[Separación entre columnas y filas]**

Aun pudiéndose separar mediante áreas vacías en “grid-template-areas” el método correcto para definir las separaciones entre las columnas y filas Grid es el uso de estos dos atributos.

row-gap: 10px ;

column-gap:10px ;

Al igual que el resto de propiedades de Grid, las medidas pueden ser definidas en porcentajes o pixeles, siendo estos últimos los recomendados.

row-gap:

Este atributo, define la separación entre las filas del Grid.

row-gap: 10px ;

En este ejemplo estaríamos definiendo una separación de 10 pixeles entre cada una de las filas que constituyeran nuestro Grid.

column-gap:

Este atributo, define la separación entre las columnas del Grid.

column-gap:10px ;

En este ejemplo estaríamos definiendo una separación de 10 pixeles entre cada una de las columnas que constituyeran nuestro Grid.

[20-11-22 2:38]

22.3 Definir correspondencia área**[Asignar áreas a elementos]**

El paso previo a la definición del propio grid, es la asignación de áreas a elementos existentes en nuestro html.

Donde:

Elemento = cualquier [Id] o [Class] presente en nuestro documento .html

La asignación de un área a un elemento se realiza en el CSS, en el interior del mismo selector donde hemos definido el Grid. El orden es indiferente; se puede indicar antes de definir el grid o después de ello.

Este sería el atributo:

.noticia_01 {

grid-area: noticia_01;

}

En este ejemplo el elemento a “transformar” en área sería el correspondiente a class=“noticia_01”.

Es importante destacar que un elemento de área, también puede ser declarado display Flex.

Se tienen que declarar tantas “grid-area” como celdas o espacios vayamos a emplear en nuestro grid. Los elementos no declarados como “grid-area” no pueden ser ubicados en el interior del grid.

22.4 Definir forma del grid

[Asignar áreas a zonas del grid]

Tras los pasos anteriores:

- Definir el elemento contenedor como **[display:grid]**
- Definir el número de columnas mediante **[grid-template-columns]**
- Definir el número de filas mediante **[grid-template-rows]**
- Asignar elementos a áreas del grid **[grid-area]**

Finalizaríamos el grid, definiendo el “mapa” de asignación de área a zona del grid. Para ello emplearíamos el siguiente atributo. **[grid-template-areas]**

```
.contenedor{
display:grid;

grid-template-columns: 3,33%,33%,33%;

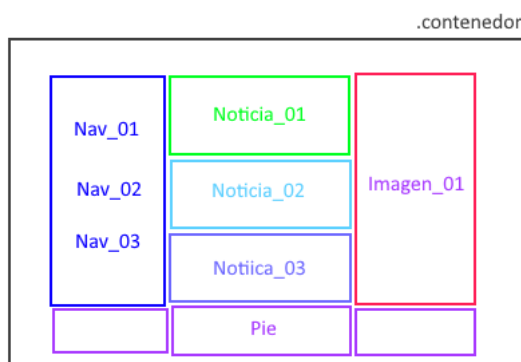
grid-template-rows: repeat(4,75px);

grid-template-areas:

“nav_01 noticia_01 imagen_01”
“nav_02 noticia_02 imagen_01”
“nav_03 noticia_03 imagen_01”
“ . pie .”;
}
```

En este caso estaríamos definiendo un grid de 3 columnas (espacio total del 99%) y 4 filas de 75px cada una.

Tendría una imagen similar a esta:



Donde:

El “.” Representa un espacio vacío en el grid.

nav_01 = elemento class – id definido como “nav_01” anteriormente asignado a “area-grid”.

nav_02 = elemento class – id definido como “nav_02” anteriormente asignado a “area-grid”.

nav_03 = elemento class – id definido como “nav_03” anteriormente asignado a “area-grid”.

noticia_01 = elemento class – id definido como “noticia_01” anteriormente asignado a “area-grid”.

noticia_02 = elemento class – id definido como “noticia_02” anteriormente asignado a “area-grid”.

noticia_03 = elemento class – id definido como “noticia_03” anteriormente asignado a “area-grid”.

Imagen_01 = elemento class-id definido como “imagen_01” anteriormente asignado a “area-grid”:

Pie = elemento class-id definido como “pie” anteriormente asignado a “area-grid”.

Tras el atributo, cada pareja de comillas [“”] corresponde a una fila de la que hemos declarado en **[grid-template-rows]**. Cada uno de los elementos que introducimos [anteriormente definidos “grid-area”] corresponden a una de las columnas que hemos declarado en **[grid-template-columns]**. La última de las filas va precedida de [;].

22.5 Método alternativo de asignar áreas al grid

A la hora de definir la forma y áreas de un grid podemos emplear el **[grid-template-areas]** o por el contrario al igual que en las hojas de cálculo, marcar una celda de “origen” y un “rango” hasta donde se expandiría el contenido de la celda de origen.

Esto se realiza mediante el siguiente atributo:

[Para definir áreas entre filas]

grid-row-start: 0;

grid-row-end: 3;

Ejemplo:

```
.logo {
  grid-area: logo;
  grid-row-start: 0;
  grid-row-end: 3;
}
```

En este caso estaríamos definiendo la clase “logo” como grid-area, marcando su inicio en la fila 0 y su expansión de contenido hasta la fila 3.

Al igual que el resto de selectores del CSS, este se ejecuta en “cascada”, de modo que si esta clase está definida posteriormente al contenedor “grid” se aplicaría esta asignación de area ignorando la que estuviera definida en **[grid-template-areas]**. La asignación preferente siempre es la última que ha sido declarada en el CSS.

Este método también tiene un atributo para definir columnas mediante un rango;

grid-column-start: 0;

grid-column-end: 3;

Funciona exactamente igual que el método anterior, salvo que, en este caso, en lugar de expandir el contenido en horizontal, la expansión se realiza en vertical.

20.6 Anexo

[Ejemplo gráfico grid]

En HTML:

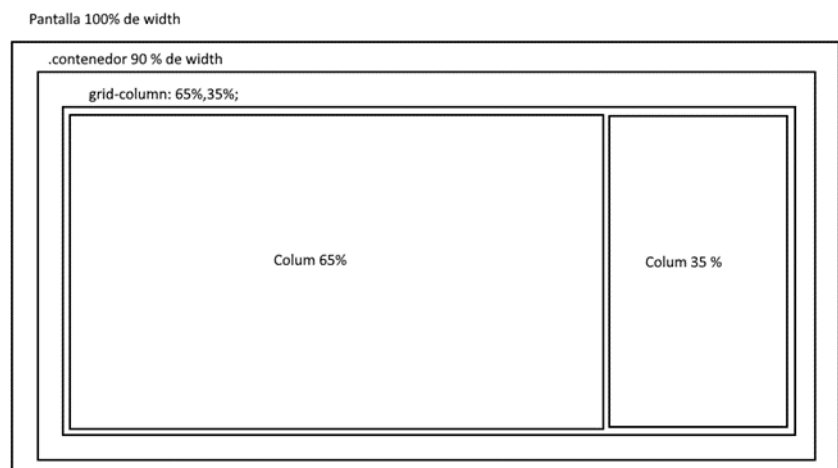
```
<main class="contenedor">
</main>
```

En CSS:

```
main {
  width: 90%;
  margin: auto;
}
```

Definición del Grid:

```
.contenedor {
  display: grid;
  grid-column: 65%,35%;
}
```



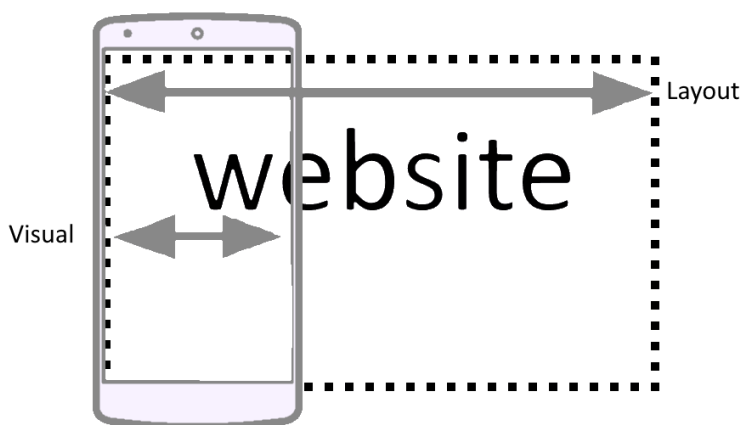
23-Adaptative Web**[Concepto Viewport]**

Originariamente el Viewport, es el area de la pantalla en la que el navegador puede renderizar contenido, es decir, el espacio disponible para mostrar el contenido de la página web.

Desktop - Viewport = Pantalla del navegador.

Layout – Viewport = Area de pantalla en CSS.

Visual –Viewport = Area de pantalla en Móvil y tablets.

**23.1 Breakpoint:****[Puntos de ruptura]**

Los breakpoints, son los puntos en los que una adaptative web empieza a aplicar los cambios que la adapten al nuevo tamaño de pantalla.

Los breakpoint por defecto son:

<576px (pantallas pequeñas)

576px-768px (móviles apaisados)

768px -992px (tablets)

992px-1200px (desktops)

>1200px (pantallas grandes)

Recomendados a la hora de codificar adaptative web:

```
@media(min-width:500px) and (max-width:768px) {
  /* pantalla móvil */
}

@media(min-width:768px) and (max-width:1200px) {
  /* pantalla Tablet */
}

@media (min-width:1200px) {
  /* pantalla PC desktop */
  /* Faltaría definir el Max... */
}
```

Metodo recomendado:

(info del ultimo examen)

```
/* @media(min-width:768px) and
(max-width:1200px) { */
```

```
@media(max-width:1200px){
```

```
/* pantalla tablet */
```

Solo hay que definir el max, no hace falta indicar el minimo.

12-04-23

23.2 Media Queries

[@media]

Los media queries son los breakpoints dentro del CSS. Es importante recordar que el correcto funcionamiento de las @media está directamente relacionado con la inclusión del archivo [normalize.css] en la cabecera del .html

El archivo de estilos normalize, adapta e iguala las proporciones del visor (cada navegador interpreta los píxeles y tamaños de un modo diferente). Si no se incluye este archivo en la cabecera del html, es posible que las @media no funcionen correctamente.

Requisitos para emplear las Media Queries:

1- Normalize.css linkeado en nuestra cabecera del HTML

2- También en nuestra cabecera:

```
<meta name="viewport" content="width=device-width,initial-scale=1.0">
```

23.2 Declaración de Media Queries

Las @media se pueden declarar en el documento principal de nuestro CSS o mediante documentos de estilo individualizados (uno por cada breakpoint).

Declaración en CSS principal:

```
@media(min-width:500px) and (max-width:768px) {  
/* pantalla móvil */  
/*aquí vendría todo el contenido CSS para la website desde 500px de ancho  
a 768px */  
}
```

Declaración mediante CSS individualizados:

```
<link rel="stylesheet" media="(max-width:576px)" href="small.css"/>  
<link rel="stylesheet" media="(max-width:768px)" href="medium.css"/>  
<link rel="stylesheet" media="(max-width:992px)" href="large.css"/>
```

Todos estos links estarían en la parte superior de nuestro documento html5, junto al resto de links de CSS.

23.3. Método de trabajo en Adaptive Web

El método correcto de trabajo en desarrollo de webs adaptativas, es tras definir los @media en el CSS comenzar a trabajar en la web de resolución más pequeña. Y tras ella proseguir con las que muestren más elementos.

23.4- Adaptación de imágenes en adaptative Web

Existen varios métodos de trabajo (no se contemplan todos en este documento) si no se tiene mucho tiempo, el modo más sencillo de tratar de adaptar las imágenes sería definiendo un tamaño "máximo". Mostrándola en base a una proporción diferente, en cada una de las @media.

Metodo_01

[Definiendo tamaño máximo]

En HTML:

```
<div>

</div>
```

En CSS:

```
div {
/*Dimensiones deseadas */
}
img {
Max-width:xxx.px;
Width:100%;
}
```

Metodo_02

[Definiendo escala máxima]

```
<div class="container">

</div>
```

Con este método se respeta la escala, cargando una imagen diferente en base al tamaño que necesite renderizar el navegador.

23.5- Uso de tamaños de letra adaptativa

El elemento mas complejo de emplear en la maquetación de la web adaptativa es el de la fuente de texto. Si se define con normalidad (mediante un tamaño fijo en Px) simplemente se desborda o no es legible.

Para poder solucionar este inconveniente hay que emplear la unidad de tamaño [vw] este tipo de escala la calcula el navegador en base al tamaño del Viewport del usuario.

Ejemplo

```
.noticia_00 {
font-size:1.5vw;
}
```

Tamaños de fuente adaptativa recomendados:

Textos normales:	1.5vw;
Textos h1:	4.5vw;
Textos h2:	2.5vw;

La cantidad máxima de texto que se recomienda mostrar en pantalla, no debe superar nunca los 60-80 caracteres. Para calcular correctamente los que se muestran en adaptative se recomienda utilizar [calc()]

24- Inclusión de Java script en documentos HTML5

Antes de linkear el documento donde estén contenidos los scripts que emplearemos es importante crear un folder para ello **[/js]**

Una vez disponemos del directorio y del archivo de script **[/js/script.js]** procederíamos a cargarlo en nuestro documento html5.

```
<script src="js/script.js"></script>  
</body>  
</html>
```

Importante:

En las anteriores versiones de HTML los scripts se cargaban en el <head> en HTML5 solo funcionan los scripts si se definen al final del </body>

[Final documentación 1 evaluación AW SMX2 27-11-22 20:53]

25 – Operadores en JavaScript

Los operadores lógicos y matemáticos en JavaScript poseen un comportamiento diferente al habitual en el resto de lenguajes. Los símbolos comúnmente empleados en C++ , C# y darkbasic profesional, así como sus métodos, no son del todo compatibles en JavaScript.

Nombre	Operador abreviado	Significado
Asignación	<code>x = y</code>	<code>x = y</code>
Asignación de adición	<code>x += y</code>	<code>x = x + y</code>
Asignación de resta	<code>x -= y</code>	<code>x = x - y</code>
Asignación de multiplicación	<code>x *= y</code>	<code>x = x * y</code>
Asignación de división	<code>x /= y</code>	<code>x = x / y</code>
Asignación de residuo	<code>x %= y</code>	<code>x = x % y</code>
Asignación de exponenciación	<code>x **= y</code>	<code>x = x ** y</code>
Asignación de desplazamiento a la izquierda	<code>x <<= y</code>	<code>x = x << y</code>
Asignación de desplazamiento a la derecha	<code>x >>= y</code>	<code>x = x >> y</code>
Asignación de desplazamiento a la derecha sin signo	<code>x >>>= y</code>	<code>x = x >>> y</code>
Asignación AND bit a bit	<code>x &= y</code>	<code>x = x & y</code>
Asignación XOR bit a bit	<code>x ^= y</code>	<code>x = x ^ y</code>
Asignación OR bit a bit	<code>x = y</code>	<code>x = x y</code>
Asignación AND lógico	<code>x &&= y</code>	<code>x && (x = y)</code>
Asignación OR lógico	<code>x = y</code>	<code>x (x = y)</code>
Asignación de anulación lógica	<code>x ??= y</code>	<code>x ?? (x = y)</code>

Atención:

Asignación “residuo”, es una división en la cual se adicionan los enteros del producto. O lo que es lo mismo; una división entre X e Y en la que se suprimen los decimales del cociente.

25_01 – Operadores en JavaScript**[Adición - Resta]**

Operador unario. Agrega uno a su operando. Si se usa como operador prefijo (++x), devuelve el valor de su operando después de agregar uno; si se usa como operador sufijo (x++), devuelve el valor de su operando antes de agregar uno.

```
++x;
```

```
x++;
```

Ejemplo:

Si x es 3, ++x establece x en 4 y devuelve 4, mientras que x++ devuelve 3 y, solo entonces, establece x en 4.

Operador unario. Resta uno de su operando. El valor de retorno es análogo al del operador de incremento.

```
--x;
```

```
x--;
```

Ejemplo:

Si x es 3, entonces --x establece x en 2 y devuelve 2, mientras que x-- devuelve 3 y, solo entonces, establece x en 2.

25_02 – Operadores en JavaScript**[Asig.adición – Asig. Resta]**

Mediante estos símbolos, manteniendo el valor inicial de X adicionamos la variable Y.

```
x += y
```

Ejemplo:

Si x valiera [2] e Y tuviera el valor de [5]; x valdría (tras esta asignación) 7.

O lo que es lo mismo:

```
x = x + y
```

Mediante estos símbolos, manteniendo el valor inicial de X restamos la variable Y.

```
x -= y
```

Ejemplo:

Si x valiera [5] e Y tuviera el valor de [2]; x valdría (tras esta asignación) 3.

O lo que es lo mismo:

```
x = x - y
```

25_03 – Operadores en JavaScript

[Operadores de comparación]

Operador	Descripción	Ejemplos que devuelven true
Igual (==)	Devuelve true si los operandos son iguales.	<pre>3 == var1 "3" == var1 3 == '3'</pre>
No es igual (!=)	Devuelve true si los operandos <i>no</i> son iguales.	<pre>var1 != 4 var2 != "3"</pre>
Estrictamente igual (===)	Devuelve true si los operandos son iguales y del mismo tipo.	<pre>3 === var1</pre>
Desigualdad estricta (!==)	Devuelve true si los operandos son del mismo tipo pero no iguales, o son de diferente tipo.	<pre>var1 !== "3" 3 !== '3'</pre>
Mayor que (>)	Devuelve true si el operando izquierdo es mayor que el operando derecho.	<pre>var2 > var1 "12" > 2</pre>
Mayor o igual que (>=)	Devuelve true si el operando izquierdo es mayor o igual que el operando derecho.	<pre>var2 >= var1 var1 >= 3</pre>
Menor que (<)	Devuelve true si el operando izquierdo es menor que el operando derecho.	<pre>var1 < var2 "2" < 12</pre>
Menor o igual (<=)	Devuelve true si el operando izquierdo es menor o igual que el operando derecho.	<pre>var1 <= var2 var2 <= 5</pre>

25_03 – Operadores en JavaScript**[Operadores de comparación]****Atención:**

El operador 'menor qué' junto a 'mayor qué' se suelen emplear en operaciones condicionales como métodos de 'igualdad'.

25_04 – Operadores en JavaScript**[Operadores Aritméticos]**

Un operador aritmético toma valores numéricos (ya sean literales o variables) como sus operandos y devuelve un solo valor numérico. Los operadores aritméticos estándar son suma (+), resta (-), multiplicación (*) y división (/). Estos operadores funcionan como lo hacen en la mayoría de los otros lenguajes de programación cuando se usan con números de punto flotante.

Operador	Descripción	Ejemplo
Residuo (%)	Operador binario. Devuelve el resto entero de dividir los dos operandos.	12 % 5 devuelve 2.
Incremento (++)	Operador unario. Agrega uno a su operando. Si se usa como operador prefijo (++x), devuelve el valor de su operando después de agregar uno; si se usa como operador sufijo (x++), devuelve el valor de su operando antes de agregar uno.	Si x es 3, ++x establece x en 4 y devuelve 4, mientras que x++ devuelve 3 y, solo entonces, establece x en 4.
Decremento (--)	Operador unario. Resta uno de su operando. El valor de retorno es análogo al del operador de incremento.	Si x es 3, entonces --x establece x en 2 y devuelve 2, mientras que x-- devuelve 3 y, solo entonces, establece x en 2.
Negación unaria (-)	Operador unario. Devuelve la negación de su operando.	Si x es 3, entonces -x devuelve -3.
Positivo unario (+)	Operador unario. Intenta convertir el operando en un número, si aún no lo es.	+"3" devuelve 3. +true devuelve 1.
Operador de exponenciación (**)	Calcula la base a la potencia de exponente, es decir, baseexponente	2 ** 3 returns 8. 10 ** -1 returns 0.1.

25_05 – Operadores en JavaScript**[Operadores Lógicos]**

Los operadores lógicos se utilizan normalmente con valores booleanos (lógicos); cuando lo son, devuelven un valor booleano. Sin embargo, los operadores `&&` y `||` en realidad devuelven el valor de uno de los operandos especificados, por lo que, si estos operadores se utilizan con valores no booleanos, pueden devolver un valor no booleano. Los operadores lógicos se describen en la siguiente tabla.

Operador	Uso	Descripción
AND lógico (<code>&&</code>)	<code>expr1 && expr2</code>	Devuelve <code>expr1</code> si se puede convertir a <code>false</code> ; de lo contrario, devuelve <code>expr2</code> . Por lo tanto, cuando se usa con valores booleanos, <code>&&</code> devuelve <code>true</code> si ambos operandos son <code>true</code> ; de lo contrario, devuelve <code>false</code> .
OR lógico (<code> </code>)	<code>expr1 expr2</code>	Devuelve <code>expr1</code> si se puede convertir a <code>true</code> ; de lo contrario, devuelve <code>expr2</code> . Por lo tanto, cuando se usa con valores booleanos, <code> </code> devuelve <code>true</code> si alguno de los operandos es <code>true</code> ; si ambos son falsos, devuelve <code>false</code> .
NOT lógico (<code>!</code>)	<code>!expr</code>	Devuelve <code>false</code> si su único operando se puede convertir a <code>true</code> ; de lo contrario, devuelve <code>true</code> .

Ejemplos de expresiones que se pueden convertir a `false` son aquellos que se evalúan como `null`, `0`, `NaN`, la cadena vacía (`""`) o `undefined`.

Ejemplos:**[`&&` (AND lógico)]**

```
var a1 = true && true;    // t && t devuelve true
var a2 = true && false;   // t && f devuelve false
var a3 = false && true;    // f && t devuelve false
var a4 = false && (3 == 4); // f && f devuelve false
var a5 = 'Cat' && 'Dog';   // t && t devuelve Dog
var a6 = false && 'Cat';   // f && t devuelve false
var a7 = 'Cat' && false;   // t && f devuelve false
```

25_06 – Operadores en JavaScript**[Condicional ternario]**

El operador `condicional` es el único operador de JavaScript que toma tres operandos. El operador puede tener uno de dos valores según una condición. La sintaxis es:

```
condition ? val1 : val2
```

Si `condition` es `true`, el operador tiene el valor de `val1`. De lo contrario, tiene el valor de `val2`. Puedes utilizar el operador condicional en cualquier lugar donde normalmente utilizas un operador estándar.

Ejemplo:

```
var status = (age >= 18) ? 'adult' : 'minor';
```

Esta declaración asigna el valor `"adult"` a la variable `status` si `age` es de dieciocho años o más. De lo contrario, asigna el valor `"minor"` a `status`.

25_07 Resto de operadores en JavaScript

Los operadores contemplados en estos apuntes, no representan la totalidad existente en JavaScript. Además de los mencionados, existen los ‘operadores lógicos de bit’ y los ‘operadores bit a bit’ (entre otros).

Los presentes en estos apuntes, son los requeridos para la asignatura de AW de SMX. Para más información consulte:

https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Expressions_and_Operators

26 – Tipos y declaración de datos

String

Se trata de un dato 'String' para java script cada palabra o string es un array. Este mismo dato correspondería a:

```
let dato;  
dato = "hola";
```

dato[i] y contendría 3 registros {h,o,l,a}

Al igual que en los lenguajes tipificados, se puede asignar durante la declaración.

```
[let dato ="hola";]
```

Number

Se trata de un dato 'Number'. En javascript solo hay un tipo de dato numérico, por lo que es indiferente si es decimal o no

```
let dato;  
dato = 100;
```

Se puede iniciar durante la declaración (como los strings) y se puede verificar con 'typeof dato'

Boolean

Se trata de un dato 'Boolean' este dato solo podría tener dos valores [true-false]

```
let dato;  
dato = true;
```

Al igual que en los lenguajes tipificados, se puede asignar durante la declaración. *Posee un método especial en las funciones condicionales.

Null

Este tipo de dato, no siempre es producto de un "error" existen datos nulos, sin valor alguno.

```
NULL  
NULL
```

Suelen producirse cuando se interrumpe una función sin que esta llegue a concluir. También cuando se solicita el 'return' de un dato que o no existe o no a llegado a ser producido por un error o final abrupto 'break' 'throw' o similares.

26 – Tipos y declaración de datos**[Continuación]****Const**

Se trata de un dato 'const' estos datos tienen una serie de particularidades, que les hace diferentes frente al resto de tipos.

```
const DATO = 7;
```

- 1- Se tienen que inicializar desde su declaración, no hacerlo provoca un error.
- 2- No se pueden volver a inicializar (reasignar valor) hacerlo provoca un error.
- 3- Una 'const' no puede compartir nombre ni con otro dato ni con una función.
- 4- Es un dato inmutable, no se puede modificar de ningún modo.
- 5- Se recomienda identificarlos haciendo uso de las mayúsculas.

Undefined

Se trata de un dato 'Undefined', se produce cuando se declara un dato y se pide información del mismo (o interacción) sin haberlo inicializado.

```
let dato;
console.log(type
of dato);
```

Este error se puede evitar inicializando la variable desde su declaración.

```
[let dato ="hola";]
'dato'se definiria como 'string'
```

Object

Se trata de un dato tipo 'object' el equivalente a 'struct' o 'type' de otros lenguajes. El objeto es una estructura que a su vez contiene sub-datos, el acceso a los mismos siempre viene precedido del identificador del objeto seguido de un punto:

```
var persona = {
nombre:"Pedro",
edad: "38",
clase: "aw" };
```

```
dato = persona.nombre;
asignaría el string 'Pedro' a la variable
[dato]
```

27 - Método Array en Javascript

Existen dos métodos de 'Array' en javascript; el primero [estático] es el más sencillo, y permite control total sobre el puntero de datos.

```
var
array_casero[
registro_00,registro_01];
```

El segundo método [dinámico] es forzar el modo 'push' originalmente empleado en formularios, para introducir un dato desde la última posición del array.

Ambos métodos son correctos, aunque no ofrecen el mismo grado de control sobre los datos.

27 - Método Array en Javascript**[Continuación]**

El siguiente array se declara de una sola vez, mediante variables definidas previamente.

Ejemplo:**[Array estatico]**

```
var registro_00;
var registro_01;
var registro_02;
var registro_03;

registro_00 = prompt("introduce valor del registro 01",registro_00);
registro_01 = prompt("introduce valor del registro 02",registro_01);
registro_02 = prompt("introduce valor del registro 03",registro_02);
registro_03 = prompt("introduce valor del registro 04",registro_03);

var array_casero[registro_00,registro_01,registro_02,registro_03];
```

Declaración de arrays estáticos:

```
let asignaturas = ['aw','eie','sox','sx','si'];
```

Lectura de dato:

```
Resultado = asignatura[2];
```

Declaración de arrays dinámicos:

```
let asignaturas = [];
```

Introducción de dato:

```
Asignaturas.push('dato');
```

27 - Método Array en Javascript**[Continuación]****Ejemplo:****[Array dinámico]**

Este sería el método [push] para arrays en javascript:

```
let array = [];  
  
function array_dinamico() {  
  
    let n_registros;  
  
    let s_dato_base;  
  
    n_registros = prompt("¿Cuántos registros quiere  
introducir?",n_registros);  
  
    for (conta = 0; conta < n_registros; conta++) {  
  
        s_dato_base= "";  
  
        s_dato_base = prompt("introduzca el registro:"+" "+conta+"  
"+" ":"",s_dato_base);  
  
        array.push(s_dato_base);  
  
    }  
  
    document.write ("dimension del array:"+array.length);  
  
    for(conta = 0; conta < array.length;conta++) {  
  
        document.write("registro_array:"+  
"+conta+"cocontenido:"+array[conta]);  
  
    }  
  
    return array;  
  
}
```

28-Bucles [básicos]

```
do{  
}while(condición  
);
```

Este sería el bucle tipo 'do' este bucle repite su contenido, hasta que se cumple la condición indicada en 'while'.

Ejemplo de bucle 'do':

```
function bucle_do() {  
  
    let conta = 1;  
  
    do {  
  
        alert("iteración:" + conta);  
  
        conta ++;  
  
    }while(conta != 5);  
  
}
```

Repite bucle, hasta que la variable de inc llega a '5'.

```
while(condición)  
{  
};
```

Este tipo de bucle 'while' repite su interior mientras la condición no se cumpla.

El siguiente ciclo del while se repite siempre que 'n' sea menor que 3:

```
function bucle_while(){  
  
    let n = 0;  
  
    let x = 0;  
  
    while (n < 3) {  
  
        n++;  
  
        alert("valor de x:"+""+x+""+"valor de  
n:"+""+n);  
  
        x += n;  
  
    }  
}
```

28-Bucles [básicos]**[Continuación]**

Este tipo de bucle es el que permite mayor control sobre los ciclos. En el momento de su definición se inicializa la variable de control, su condición de salida y su modificación por ciclo.

```
for([inicialización de variable control],[condición de finalización],[modificador de variable de control]){ };
```

Bucle 'for' este método es muy parecido al de C/C++:

```
function bucle_for(){  
  
    let limite;  
  
    limite = 3;  
  
    for(conta = 0; conta < 3; conta++) {  
  
        alert("valor del contador:"+" "+conta+"  
        "+"limite:"+" "+limite);  
  
    };  
  
}
```

Funcionamiento:

```
for([inicialización de variable control], [condición de finalización],  
    [modificador de variable de control]){};
```

Importante:

El bucle for, se repetirá siempre y cuanto su condicional tenga el valor 'true'.

Ejemplo:

```
For ( i = 0 ; i < conta ; i++){}
```

En este caso el bucle se repetiría mientras la variable 'conta' sea superior al valor del puntero 'i'.

29- Condicionales / Switches

```
if (condición) {  
  sentencia1 }  
  else {  
    sentencia2 }
```

El condicional más básico es el método 'if', al iniciarse valora el resultado de la condición, si es afirmativo ejecuta el código de sentencia1.

Opcionalmente se puede añadir un resultado "alternativo" o 'else', instrucciones que solo se ejecutaran si no se cumple la condición inicial.

```
function condicional(){  
  
  let numero;  
  
  numero = parseInt(prompt("introduce un  
numero...",numero));  
  
  if (!isNaN(numero)){  
  
    if(numero >= 1){  
  
      alert("El numero introducido es igual o mayor de  
1");  
  
    }else {  
  
      alert("El numero introducido es menor de 1");  
  
    }  
  
  } else {  
  
    alert("El valor introducido no era un número");  
  
  }  
  
  }  
  
  // isNaN() siempre devuelve "false" si el dato es  
  numérico.
```

29- Condicionales / Switches**[Continuación]**

```
switch(variable) {  
  case 01:  
    break;  
  case 02:  
    break;  
  case 03:  
    break;  
  default:  
    -sentencia-  
}
```

El método 'switch' permite auditar el valor de una variable y en base a ello realizar una acción determinada.

Este switch, revela el nombre del mes que corresponde a un número entre 1 y el 3.

```
function sw(){  
  let mes;  
  
  mes = parseInt(prompt("introduzca el número del  
mes [entre 01 y 03]",mes));  
  
  if(!isNaN(mes)){  
    switch(mes){  
      case 01:  
        alert("Ese mes es enero");  
        break;  
      case 02:  
        alert("Ese mes es febrero");  
        break;  
      case 03:  
        alert("Ese mes es marzo");  
        break;  
      default:  
        alert("Ese número de mes no existe");  
        break;  
    }  
  }else { alert("Eso es una palabra no un número");  
  }  
}
```

29- Condicionales / Switches**[Continuación]**

Este método permite "anidar" 'cases' haciendo posibles varias respuestas a modo de desencadenante.

Este switch, tiene anidamiento de 'case' teniendo múltiples respuestas validas por caso.

```
function sw_02(){  
    let cont_var;  
  
    cont_var = prompt("Elige uno de estos elementos  
[diamante,cuarzo] o [manzana, pera]",cont_var);  
  
    switch(cont_var){  
  
        case "diamante":  
  
        case "cuarzo":  
  
            alert(cont_var + " " + "Es una clase de cristal");  
  
            break;  
  
        case "manzana":  
  
        case "pera":  
  
            alert(cont_var + " " + "Es una clase de fruta");  
  
            break;  
  
        default:  
  
            alert("No has introducido ninguno de los elementos  
seleccionables.");  
  
            break;  
  
    }  
  
}
```

30- Funciones [básicas]

```
function
nombre(parámetro
s){
return
variable;}
```

Existen tres métodos de función en javascript. Función simple, parametrizada y recursiva.

Función con doble parámetro [n = un número, nombre = una palabra]

```
//ejecutor: onclick ="prueba(2,'hola')"
```

 [En un button de HTML]

```
function funcion_01(n,nombre){

calculo = (n * n);

alert("El resultado de la función:"+" "+ calculo+"
"+nombre);

}
```

No es necesario inicializar las variables incluidas a modo de parámetros. Se consideran datos "locales". Si se desean sacar de la función se tiene que hacer mediante el 'return' al final de la misma.

Las variables 'string' se tienen que declarar desde el html con comillas simples ' de lo contrario inutilizan el script.

Este ejemplo muestra la recursividad de función y la llamada a función para inicializar una variable.

```
function funcion_02() {

prueba_00 = joor(4);

alert("este es el valor de retorno de la
función:"+" "+prueba_00)

}

function joor(n) {

resultado = n * n;

return resultado;

}
```

[0:50 17-12-22]

31- Objetos**[Avanzado]**

En JavaScript existen tres métodos de creación de objetos. A la hora de definirlos cualquiera de estos métodos es correcto.

Los métodos a su vez poseen dos características:

Atributos Son partes del propio objeto, se accede a ellos mediante el nombre del objeto seguido de un punto.

`Nombre_objeto.atributo;`

Métodos Son funciones asociadas al objeto. Se accede a ellos del mismo modo que en los atributos, eso sí, seguido de un par de paréntesis (y los parámetros necesarios en caso de ser requeridos).

`Nombre_objeto.metodo();`

Objetos declarativos o literales:

Son los objetos más básicos y sencillos de declarar. Se basa en la declaración de una variable, cada uno de los atributos y métodos se asignan mediante un 'nombre' y dos puntos.

Ejemplo:

```
Var persona1 = {  
    Nombre: 'pepito',  
    Edad: 16,  
    saludo: function() {  
        alert('hola');  
    }  
}
```

Estos atributos/métodos, están separados entre sí mediante una coma ',' estando el último de ellos, finalizado con punto y coma ';'.

31- Objetos**[Avanzado]****Objetos 'new Object':**

Son objetos prácticamente iguales a los literales, aunque en este caso se definen de un modo similar al de las funciones.

Ejemplo:

```
var persona1 = new Object({  
    nombre: 'miaus',  
    edad: 25,  
    maullido: function() {}  
});
```

Objetos contruidos:

Son los objetos más complejos y completos. Estos se crean mediante una función, accediendo a sus atributos/métodos por medio de la palabra reservada 'this'.

Ejemplo:

```
Function Persona(nombre,edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
    this.saludito = function () {  
        alert('hus hus');  
    }  
}  
  
let persona1 = new Persona ('coco',25);
```

32- Métodos de trabajo con objetos**[Constructors]**

Un objeto (independientemente del método empleado) es un elemento 'único' pero esto no quiere decir que no se pueda emplear a modo de plantilla. En este caso estaríamos hablando de 'constructores' u 'Object Constructors'.

Este método es el más empleado a la hora de manejar objetos en JavaScript y C++.

¿Cómo funciona el método constructor?

Primero tenemos que crear la estructura de un objeto, como si se tratara de un objeto único.

Ejemplo:

```
function Car(make, model, year) {  
  
    this.make = make;  
  
    this.model = model;  
  
    this.year = year;  
  
}
```

Seguidamente simplemente declaramos tantas variables como objetos necesitemos. Estos objetos obviamente, contarán con los mismos atributos y métodos que el constructor.

Ejemplo:

```
var mycar = new Car('Eagle', 'Talon TSi', 1993);  
  
var kenscar = new Car('Nissan', '300ZX', 1992);  
  
var vpgscar = new Car('Mazda', 'Miata', 1990);
```

En este caso habríamos declarado tres objetos "Car" [mycar,kenscar,vpgscar]

32- Métodos de trabajo con objetos**[Constructors]****¿Cómo accedemos a los datos de los objetos?**

La manera de acceder a los datos de cada objeto, es indicando su nombre o ID, seguido de un punto y el atributo al que deseamos acceder.

Ejemplo:

```
alert("este es el objeto_03 " + coche_03.fabricante + "
      "+coche_03.modelo + " "+ coche_03.año);
```

En este caso estaríamos accediendo a las propiedades de un objeto denominado "objeto_03". Mostrando los datos de su atributo "fabricante", "modelo" y "año".

Información adicional del método constructor:

https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Working_with_Objects

[19-12-22 0:24]

Resumen:**Acceso a atributo de objeto tras constructor:**

```
This.nombre = a ;                                >>>>      objeto.nombre
```

Acceso a método de objeto tras constructor:

```
This.total = function(){}                        >>>>      objeto.total()
```

A los métodos de objeto se accede del mismo modo que a las funciones.

Importante:

Para poder emplear los datos resultantes de una función de método, hay que colocar los datos resultantes en un return envuelto en paréntesis, esa será la respuesta del método.

```
00  this.total = function(){
01  if (parseInt(d) != 0 ) {
02  alert('Tiene un descuento de: '+' "+d+" "+"%");
03  return((parseInt(c) - ((parseInt(c)*parseInt(d))/100)));
04  }
```


33- Métodos de trabajo DOM

Los métodos 'DOM' nos permiten manipular todos los documentos que forman parte de la estructura web, todo ello mediante instrucciones en javascript.

Estos métodos engloban desde el acceso a bases de datos (SQL) a interacciones con el servidor (mediante PHP) e incluso la modificación de los estilos (CSS) o semántica de la web (HTML). En el presente curso, únicamente requerimos del método DOM para interacción con HTML – CSS.

33.1 Método DOM [HTML]

Acceso a elementos etiquetados [id - class]

Podemos acceder a cualquier elemento presente en el HTML como si fuera un "objeto". Esto quiere decir que el 'id' o el 'class' definido previamente nos permitirá acceder al objeto en base, y a sus propiedades mediante un '.'

Ejemplo:

```
<label id=nombre_02 for="Nombre">Nombre</label>

<button onclick="ejemplos()" type="button">26-12-22</button>
```

Si quisiéramos modificar el texto de esta etiqueta identificada con el id 'nombre'.. Esta función tendría el desencadenante de "onclick" de un botón previamente ubicado bajo el label del HTML.

```
function ejemplos(){

    document.getElementById('nombre_02').textContent='prueba';

}
```

Lo que estaríamos haciendo, es cambiar el valor de esa etiqueta (lo que se lee en el navegador del usuario) por la palabra 'prueba'.

33- Métodos de trabajo DOM**[Continuación]****33.02. Sintaxis del método DOM [id - class]****1- Acceso a elementos por 'id'**

```
document.getElementById('identificador').propiedad
```

Donde:

Document

Corresponde al documento HTML

.getElementById('identificador')

Corresponde al id del elemento del html.

.propiedad

A la propiedad que queremos acceder.

2-Acceso a elementos por 'class'

```
document.getElementsByClassName().length
```

Ejemplo práctico:

Acceso para captura de valor en variable:

[En HTML]

```
<input type="text" id="nombre">
```

[En JavaScript]

```
Let ejemplo_DOM = document.getElementById('nombre').value;
```

En este ejemplo estaríamos asignando el valor que el usuario a introducido en el input identificado por 'nombre' a la variable 'ejemplo_DOM'.

33- Métodos de trabajo DOM**[Continuación]**

Atributos del DOM id - Class

Etiqueta de texto:

.textContent Texto que muestra la etiqueta.

Input:

.value Texto introducido por el usuario. También se puede emplear, para introducir texto por “defecto”.

Es importante destacar que existen numerosos atributos y sub-atributos accesibles desde el método DOM. Los aquí mostrados son solo un breve ejemplo.

3-Aceso a elementos por semántica

```
document.body.style.backgroundColor = 'red';
```

En este ejemplo estaríamos accediendo al CSS correspondiente al elemento semántico ‘body’ asignándole el color de fondo ‘rojo’.

Importante:

Solo se pueden acceder mediante método DOM a las semánticas principales del documento HTML. El resto de etiquetas, simplemente son ignoradas por el navegador.

Accesibles:**Body – head**

Al resto de etiquetas se accede mediante un id o una clase , previamente definida por el usuario.

33.03. Sintaxis del método DOM [Styles]

Así como podemos acceder a cualquier elemento del documento .html también podemos hacer lo mismo con la hoja de estilos o CSS. Para ello simplemente hay que implementar ‘.style.atributo_css’ tras identificar el elemento que deseamos manipular.

33- Métodos de trabajo DOM**[Continuación]****33.03. Sintaxis del método DOM [Styles]**

Los atributos disponibles al acceder a los estilos de elemento, son exactamente los mismos que se pueden emplear en el documento CSS.

Ejemplo:

```
<input type="text" id="nombre">
```

Modificar su ancho:

```
document.getElementById('nombre').style.width='10px';
```

Modificar su color de fondo:

```
document.getElementById('nombre').style.backgroundColor='green';
```

33.04. Modificar el código de HTML por DOM

Por ultimo (requerido en este curso) el método DOM permite modificar a tiempo real el código que está cargando el navegador. Esto nos permite modificar la web a tiempo real, a nuestro antojo.

33.05 Método DOM [innerHTML]

```
document.getElementById('resultados').innerHTML =  
("<h1>" + frases[Math.round(Math.random()*4)] + "</h1>");
```

En este ejemplo, estaríamos accediendo a la sección del documento etiquetada con el id 'resultados', y en esa misma sección, introduciendo un fragmento nuevo de código HTML.

Importante:

Al emplear el método 'innerHTML' hay que recordar que todo lo referente al código HTML siempre se escribe envuelto en comillas "" y toda instrucción o variable JavaScript, simplemente se adiciona (+)

Anexo 01**[Material de examen]****34. Numeros aleatorios en JavaScript**

El manejo de los números aleatorios en javascript es diferente al que solemos ver en el resto de lenguajes. A diferencia de C++ o C# no se puede manipular el seed del random.

En su lugar, se emplea una sintaxis algo peculiar:

```
Let numero_aleatorio = (Math.random()*4);
```

La función 'math.random()' ejecuta un número entre 1 y 0 que a su vez, opera con el número que tiene a su derecha.

El método correcto de uso sería el siguiente:

```
Math.random()*[numero máximo del random]
```

¿Qué problema se puede presentar con este método?

Al entregar un número entre 1 y 0 el producto es un número con muchísimos decimales. Si lo requerimos para añadir aleatoriedad a un puntero, o a una función específica (o incluso para un RGB de color) nos veremos inmersos en infinidad de bugs debido a que el número no es legible por nuestra función.

¿Cómo se eliminan los decimales del random en Java script?

Introduciendo un método matemático que redondea los decimales hacia su entero más cercano.

```
Let numero_aleatorio = Math.round(Math.random()*4);
```

Este método se denomina en javascript 'math.round()', siguiendo el ejemplo de la línea superior, la variable 'numero_aleatorio' tendría un valor entre 1 y 4. El cero salvo muy rara ocasión, nunca sería su valor. Esto es debido al 'redondeo' hacia el entero que realiza este método.

```
trienios = Math.floor((antigüedad / 3)) * p_trienio;
```

* Math.floor redondea hacia abajo.

35. Transformar variable en Integer

O lo que es lo mismo, transformar cualquier valor en un 'número entero'. Este método es imprescindible en javascript, dada la idiosincrasia que tiene este lenguaje con el tratamiento de datos.

Para javascript todo, (absolutamente todo) es un 'string' por defecto. Esto que a la par puede parecer interesante, es un verdadero problema ante más del 99% de los scripts que tengamos que programar.

¿Cómo evitar los errores NULL - NaN?

Estos errores se dan cuando se está tratando a un string como si fuera un integer o viceversa. Para evitarlo tenemos que realizar dos acciones.

36.01-Declarar las variables desde el principio

Misteriosamente en clase no se ha dado importancia (ni se ha mencionado) este método, aunque sorprenda; es el más empleado profesionalmente. Nos ahorra muchos problemas tanto a la hora de manejar strings como números.

En lugar de :

Let variable_01;

Var variable_02;

Let variable_01 = ""; Para inicializarla como un string.

Var variable_02 = 0; Para inicializarla como un número.

Let variable_03 = true; Para inicializarla como un booleano.

Esta práctica nos evitara el error 'NaN'

36.02 Transformar strings en números enteros

parseInt();

Ejemplo:

```
let variable_01 = 0; let variable_02 = 0;

variable_01 = prompt('introduzca un numero',variable_01);

variable_02 = prompt('introduzca el siguiente numero',variable_02);

resultado = parseInt(variable_01) + parseInt(variable_02);

alert('el resultado es...'+" "+resultado);
```

Mediante el método 'parseInt()' hemos transformado las variables, en números enteros que se pueden operar.

Si prescindimos de ello, JavaScript tratará los números como si fueran caracteres (letras), uniéndolas en lugar de operarlas.

Ejemplo:

Variable_01 = 5; variable_02 = 4;

Resultado = variable_01 + variable_02 [Daria como resultado '54']

37-Verificar el tipo de dato que nos entrega el usuario

Cuando manejamos inputs (da igual el tipo), el que definamos la clase de dato realmente es irrelevante. Esto se da cuando por ejemplo pedimos un “numero” y la persona nos entrega una “letra”. Aunque hubiéramos definido el input como numérico a la hora de trabajar con el tendríamos un error ‘NaN’ al no poderse operar con las variables.

Java Script tiene dos métodos para evitar este error:

37.01 Método de comprobación ‘typeof()’

```
Let dime_que_es = typeof(variable);
```

Este método nos devuelve un string diciéndonos como identifica al dato.

Para string :	output: "string"
Para numérico:	output: "number"
Para booleano:	output: "boolean"
No definido:	output: "undefined"

¿Qué problema presenta typeof()?

A diferencia del siguiente método, ‘typeof()’ puede ser ‘trampeado’, si nosotros mismos definimos la variable tras su input, esa declaración será la que tomara el método. En otras palabras; si declaramos un input como “numérico” siendo string, ‘typeof()’ nos dirá erróneamente que es un valor numérico. Obviamente esto no suele ocurrir intencionadamente, pero puede ser un problema si no estamos muy atentos a lo que codificamos...

37.02 Método de comprobación ‘!isNaN()’

En Html:

```
<input type="text" id="edad">
```

En Javascript:

```
edad = document.getElementById('edad').value;

if (!isNaN(edad)){

}else {

Alert('No has introducido un numero, reinicia el proceso.')

}
```

37-Verificar el tipo de dato que nos entrega el usuario**[Continuación]****37.02 Método de comprobación 'isNaN()'**

Este método, a diferencia que el anterior, es capaz de dilucidar el tipo de la variable desde el momento en que el usuario la introduce en el input.

'isNaN()' es capaz incluso de distinguir cuando un string tiene números (declarándolo como string) o si una serie de números tiene algún carácter.

¿Cómo funciona isNaN()?

Este método examina la variable que le indiquemos entre sus paréntesis y devuelve una respuesta booleana en base a lo que identifica.

Si el valor de la variable es un numero: False

Si el valor de la variable es un string: True

¿Cómo aplicamos el filtro de isNaN()?

Para poder emplear correctamente este método tenemos que utilizar una condicional booleana.

```
if (!isNaN(edad)){  
  }else {  
    Alert('No has introducido un numero, reinicia el proceso.')  }  
}
```

De este modo estaremos definiendo que si el valor de 'isNaN()' es 'False' podemos proseguir[**Porque la variable 'edad' es numérica**], de lo contrario, no. Para valorar la condición booleana, solo tenemos que poner una exclamación delante del método:

!isNaN(edad)

38-Verificar la longitud de un dato

Otro de los problemas que nos podemos encontrar (sobre todo con formularios) es cuando el usuario inicia la entrega con campos 'vacíos'. Esto se puede evitar mediante una sencilla condicional, y el atributo de dato '.length'

Ejemplo:

```
nombre = document.getElementById('nombre').value;
apellidos = document.getElementById('nombre').value;
edad = document.getElementById('edad').value;

if (nombre.length != 0 && apellidos.length != 0 && edad.length != 0){
}
else{
Alert('Faltan campos por rellenar, revise el formulario');
}
```

En este ejemplo estaríamos comprobando que todos los datos tengan algún carácter, de lo contrario no continuamos con el condicional. Este código se podría mejorar, especificando incluso, el número de caracteres que a detener la variable... Los operadores empleados se encuentran al principio de este bloque de javascript.

39 – Transformar variable en string

En raras ocasiones tendremos que transformar una variable en 'string'. Este caso sobretodo está contemplado a la hora de manejar datos numéricos en textos (para informes y similares).

```
String(variable);
```

40 – Dividir un string

Si tenemos la necesidad podemos dividir un string, para generar otro. Esto se realiza mediante el método 'substring'

Ejemplo:

```
variable_01 = prompt('introduzca una palabra',variable_01);

final = variable_01.length;           [valor = hola_que_tal]

sub_string = variable_01.substring(5,parseInt(final));
```

En este caso el string que se almacenaría en la variable 'sub_string' seria: 'que_tal'

40 – Dividir un string**[Continuación]**

Es importante recordar que en JavaScript los datos siempre se inician en el número 0.

Sintaxis:

```
[variable],substring([inicio],[final]);
```

41 – Dividir un string generando un array

Este es un método algo laborioso que existe a la hora de tratar strings, y 'es el requerido para aprobar el examen final de esta asignatura' (Según comentarios del profesor).

¿Qué usos tiene este método?

Se puede emplear para poder fragmentar largas cadenas de texto, o datos, reduciendo de este modo su tamaño total y facilitando su procesamiento.

¿Cómo funciona este método?

Este método no difiere mucho del anterior (división de string) e incluso es más sencillo. Para poder manejarlo solo necesitamos identificar un carácter (o incluso espacios vacíos) para definir donde 'terminan' las palabras o fragmentos de texto que deseamos fragmentar.

```
variable.split('carácter que divide');
```

El método fragmenta la palabra o string, en trozos más pequeños, asignándolos automáticamente a un array.

Ejemplo:

```
Let variable = "Pepe - Luis - era - un - compañero - que - ya - no - está - en - clase";
```

```
El_array = variable.split('-');
```

```
alert('Tamaño del array: ' + " " + El_array.length);
```

```
alert('Registro 03: ' + " " + El_array[3]);
```

Siguiendo este ejemplo, la variable se convierte en un array de '10' registros. El carácter elegido para la fragmentación es '-'. Su tamaño sería de '10' y el registro 03 mostraría la palabra: 'un'.

[26-12-2022 22:05]

42 – Localizar un carácter en String**<case sensitive>**

Podemos localizar un carácter específico en un String mediante el método **[.include()]** Su respuesta será booleana [**True** si lo localiza, **false** si no lo localiza].

```
if (t_correo.includes("@")){ }
```

Ejemplos

[Examen 03]

43 - Función que calcula la media de dos números:

```
00  function j01(){
01      var n_num_01;
03      var n_num_02;
04      n_num_01 = parseInt(prompt("introduce el primer numero",n_num_01));
05      n_num_02 = parseInt(prompt("introduce el segundo numero",n_num_02));
06      n_resultado = ((n_num_01 + n_num_02)/2);
07      prompt("este es el resultado"+" "+n_resultado);
08      return
09  }
```

44 - Función que calcula el iva de un producto:

```
00  function j02(){
01      var n_num_03;
02      n_num_03 =  parseInt(prompt("introduce un precio",n_num_03));
03      n_iva = ((n_num_03*21)/100);
04      prompt("El iva de"+" "+n_num_03+" "+"es"+" " +n_iva);
05  }
```

03 - Función que retorna el mayor de dos números:

```
00  function j03(){
01      var n_num_04;
02      var n_num_05;
03      n_num_04 = parseInt(prompt("introduce el primer numero",n_num_04));
04      n_num_05 = parseInt(prompt("introduce el segundo numero",n_num_05));
05      n_result = Math.max (n_num_04,n_num_05);
06      prompt( "El numero mas grande de la pareja es... :" + " " +n_result);
07  }
```

Ejemplos

[Examen 03]

45 - Función que calcula el total a pagar a partir del precio de un producto según los siguientes requerimientos:

- a. Si el producto vale más de 50 € y menos o igual a 100 € se descontara 10 € euros.

```
00  function j04(){
01  var n_num_06;

02  n_num_06 = parseInt(prompt("introduce el precio del
    producto",n_num_06));

03  if (n_num_06 > 50 & n_num_06 <= 100) {
04  n_result = (n_num_06 - 10);

05  prompt("El producto tiene descuento, se queda en:" + " " + n_result);
06  } else {
07  n_result = n_num_06;

08  prompt("El producto mantiene su precio, se queda en:" + " " +
    n_result);

09  }
```

46 - Si el producto vale 50 € o menos se descontara 5 €, siempre y cuando el producto valga más de 10 € :

```
00  function j05(){
01  var n_num_07;

02  n_num_07 = parseInt(prompt("introduce el precio del
    producto",n_num_07));

03  if ( n_num_07 > 10 & n_num_07 >= 50 ) {
04  n_result = (n_num_07 - 5);

05  prompt("El producto tiene un descuento se queda en..."+" "+n_result);
06  } else {
07  n_result = n_num_07;

08  prompt("El producto no tiene descuento, se queda en..."+"
    "+n_result);

09  }
```

Ejemplos

[Examen 03]

47 - El producto que valga más de 100 € tendrá un descuento del 10% de su precio :

```
00  function j06(){
01  var n_num_08
02  n_num_08 = parseInt(prompt("introduce el precio del
    producto",n_num_08));
03  if (n_num_08 > 100) {
04  n_result = (n_num_08 - ((n_num_08 * 10) / 100) );
05  prompt("El producto tiene un descuento del 10%,quedandose en...."+"
    "+n_result)
06  } else {
07  n_result = n_num_08;
08  prompt("El producto no tiene descuento. Siendo su precio..."+"
    "+n_result)
09  }
```

48 - Función que valide si tiene un correo electrónico al tener el carácter @ y si después de este tiene un punto :

```
00  function j07(){
01  var t_correo; var t_comp_02;
02  t_correo = prompt("Introduzca una direccion de correo",t_correo);
03  if (t_correo.includes("@")){
04  t_comp_02 = t_correo.substring(t_correo.indexOf("@"));
05  if (t_comp_02.includes(".")){
06  prompt("La direccion de correo es aparentemente correcta"+"
    "+t_correo);
07  }else{
08  prompt("La direccion de correo estaba incompleta,repita el proceso.")}
10  }else {prompt("El texto introducido no es una direccion de correo.")}
11  }
```

Ejemplos

[Examen 03]

49 - Función que calcule la mediana de las notas pasadas como un parámetro en un array [estático] :

```
00  function j08 (){
01  var notas = [1,3,5,9];
02  prompt("Estos son los valores del array de notas:"+" "+notas[0]+"
    "+notas[1]+" "+notas[2]+" "+notas[3]);
03  var sum = notas.reduce((previous, current) => current += previous);
04  var avg = sum / notas.length;
05  prompt("Este es el promedio:"+" " + avg);
06  }
```

50 - Función que compare dos fechas y devuelva si son iguales o no :

```
00  function j09(){
01  var t_fecha_01;
02  var t_fecha_02;
03  t_fecha_01 = prompt("introduce primera fecha",t_fecha_01);
04  t_fecha_02 = prompt("introduce segunda fecha",t_fecha_02);
05  if (t_fecha_01 == t_fecha_02 & t_fecha_01.includes("/") &
    t_fecha_02.includes("/")){
06  prompt("Las fechas son iguales" + " " + t_fecha_01 +" "+ t_fecha_02);
07  } else {
08  prompt ("las fechas no son iguales o tienen un error de formato.
    Pruebe con DD/MM/AAAA");
09  }
```

Ejemplos

[Examen 03]

51 - Función que dado un numero entre 0 y 23 devuelva una cadena de texto con el formato "hh am" o "hh pm" . por ejemplo 13 -> 1:00 pm :

```
00  function j10(){
01  var t_hora;  var t_conversion;  var n_ok;
02  t_hora = parseInt(prompt("Introduzca un numero del 00 al
23",t_hora));
03  switch (t_hora) {
04  case 00 :
05  t_conversion = "12:00";    n_ok = "true";
06  break;
07  case 13 :
08  t_conversion = "01:00";  n_ok = "true";
09  break;
10  case 14 :
11  t_conversion = "02:00"; n_ok = "true";
11  break;
12  case 15 :
13  t_conversion = "03:00"; n_ok = "true";
14  break;
15  case 16 :
16  t_conversion = "04:00"; n_ok = "true";
17  break;
18  case 17 :
19  t_conversion ="05:00"; n_ok = "true";
20  break;
```

Ejemplos

[Examen 03]

51- Función que dado un numero entre 0 y 23 devuelva una cadena de texto con el formato "hh am" o "hh pm" . por ejemplo 13 -> 1:00 pm : [Continuación]

```
21     case 18 :
22         t_conversion = "06:00"; n_ok = "true";
23         break;
24     case 19 :
25         t_conversion = "07:00"; n_ok = "true";
26         break;
27     case 20 :
28         t_conversion = "08:00"; n_ok = "true";
29         break;
30     case 21 :
31         t_conversion = "09:00"; n_ok = "true";
32         break;
33     case 22 : t_conversion = "10:00"; n_ok = "true";
34         break;
35     case 23 :
36         t_conversion = "11:00"; n_ok = "true";
37         break;
38     default:
39         prompt("El formato introducido no es correcto. introduzca un numero
del 00 al 23");
40     }
41     if (n_ok == "true") {
42         prompt("La conversion es:"+" "+ t_conversion);
43     }
44 }
```


Ejemplos

[Examen 03]

52 - Función que según el año de nacimiento devuelva si se es "generación X". "Milenial", "zoomer" :

```
00  function j11() {
01  var n_fnacimiento;
02  n_fnacimiento = prompt("Introduce el año de tu
    nacimiento...",n_fnacimiento);
03  if (parseInt(n_fnacimiento) <= 1962 | parseInt(n_fnacimiento) >= 2022)
    {
04  prompt("No poseo datos sobre esa generacion..." );
05  }
06  if (parseInt(n_fnacimiento) >= 1963 & parseInt(n_fnacimiento) <= 1981)
    {
07  prompt("Pertenece a la generacion X , nacido entre 1963 y 1981" );
08  }
09  if (parseInt(n_fnacimiento) >= 1981 & parseInt(n_fnacimiento) <= 1993)
    {
10  prompt("Pertenece a la generacion millennial , nacido entre 1981 y
    1993" );
11  }
12  if (parseInt(n_fnacimiento) >= 2010 & parseInt(n_fnacimiento) <= 2022)
    {
13  prompt("Pertenece a la generacion Zoomers , nacido entre 2010 y 2022"
    );
14  }
15  }
```

Ejemplos

[Examen 03]

53 – Función que devuelva el precio final de la compra de un artículo / producto dependiendo de los siguientes supuestos; Si compra dos unidades se tiene que aplicar un 10 % de descuento :

```
00  function j12() {
01  var n_cantidad; var n_result; var n_subtotal;
02  n_cantidad = prompt("Cocalola, 10.50 und ; ¿Que cantidad es, 1 o 2
    unidades?",n_cantidad);
03  n_subtotal = (10.50 * parseInt(n_cantidad)) ;
04  if (parseInt(n_cantidad) >= 2) {
05  n_result = (n_subtotal - ((n_subtotal*10)/100)) ;
06  prompt("Por este numero de unidades tendra un descuento del 10%"+
    "+n_result ) ;
07  }else {
08  prompt("Por esa cantidad no tiene descuento, total:" + " "+
    n_subtotal);
09  }
```

54 - Si se compran tres unidades se tiene que aplicar un 15% de descuento:

```
00  function j13() {
01  var n_cantidad; var n_result; var n_subtotal;
02  n_cantidad = prompt("Cocalola, 10.50 und ; ¿Que cantidad es, 1,2 o 3
    unidades?",n_cantidad);
03  n_subtotal = (10.50 * parseInt(n_cantidad)) ;
04  if (parseInt(n_cantidad) >= 3) {
05  n_result = (n_subtotal - ((n_subtotal*15)/100)) ;
06  prompt("Por este numero de unidades tendra un descuento del 15%"+
    "+n_result ) ;
07  }else {
08  if (parseInt(n_cantidad) == 2) {
09  n_result = (n_subtotal - ((n_subtotal*10)/100)) ;
```

Ejemplos

[Examen 03]

54 - Si se compran tres unidades se tiene que aplicar un 15% de descuento: [Continuación]

```
10    prompt("Por este numero de unidades tendra un descuento del 10%"+
        "+n_result ) ;
11    }
12    }
13    if (parseInt(n_cantidad) == 1) {
14        prompt("Por esa cantidad no tiene descuento, total:" + " "+
            n_subtotal);
15    }
```

55- Si se compran cuatro unidades o más se tiene que aplicar un 20% de descuento:

```
00    function j14() {
01        var n_cantidad; var n_result; var n_subtotal;
02        n_cantidad = prompt("Cocalola, 10.50 und ; ¿Que cantidad es, 1,2,3 o 4
            unidades?",n_cantidad);
03        n_subtotal = (10.50 * parseInt(n_cantidad)) ;
04        if (parseInt(n_cantidad) >= 4) {
05            n_result = (n_subtotal - ((n_subtotal*20)/100)) ;
06            prompt("Por este numero de unidades tendra un descuento del 20%"+
                "+n_result ) ;
07        }
08        if (parseInt(n_cantidad) == 3) {n_result = (n_subtotal -
            ((n_subtotal*15)/100)) ;
09            prompt("Por este numero de unidades tendra un descuento del 15%"+
                "+n_result ) ;
10        }
11        if (parseInt(n_cantidad) == 2) {n_result = (n_subtotal -
            ((n_subtotal*10)/100)) ;
12            prompt("Por este numero de unidades tendra un descuento del 10%"+
                "+n_result ) ;
13        }
```

Ejemplos

[Examen 03]

55- Si se compran cuatro unidades o más se tiene que aplicar un 20% de descuento:
[Continuación]

```
14     if (parseInt(n_cantidad) == 1) {  
15         prompt("Por esa cantidad no tiene descuento, total:" + " "+  
                n_subtotal);  
16     }  
17 }
```

[30-12-22 21:13]

56 - Ocultar elemento en HTML, mostrándolo después mediante interacción en JavaScript por método DOM:

En html:

```
00     <button class="ejercicio_03" type="button"  
      onclick="ejer_03(0)">Ejercicio_03</button>  
01     <article id="e3_botones">  
02         <form>  
03             <fieldset>  
04                 <legend>[ejercicio 03 de examen prueba]</legend>  
05                 <button id="colores" type="button" onclick="ejer_03(1)"> cambiar  
                  colores letras </button>  
06                 <button id="textos" type="button" onclick="ejer_03(2)" > cambiar  
                  textos </button>  
07                 <button id="fondos" type="button" onclick="ejer_03(3)">cambiar color  
                  fondo</button>  
08             </fieldset>  
09         </form>  
10     </article>
```

Ejemplos**[Examen 03]**

56 - Ocultar elemento en HTML, mostrándolo después mediante interacción en JavaScript por método DOM: **[Continuación]**

En CSS:

```
00    #e3_botones{  
01    visibility: hidden;  
01    }
```

En javascript:

```
00    function ejer_03(n){  
01    document.getElementById('e3_botones').style.visibility = "visible";  
02    }
```

[01-01-23 23:32]

57 - Añadir parámetros condicionales en If :

```
00     prime = prompt('introduce el primer dato',prime);
01     segundo = prompt('ntroduce el segundo dato',segundo);
02     if (isNaN(prime) == isNaN(segundo)){
03         alert('ambos tienen el mismo formato');
04     } else {
05         alert('cada uno tiene un formato diferente');
06     }
07     if ( parseInt(prime) > 1 && parseInt(segundo) < 10 ){
08         alert(' el primero es mayor que 1 y el segundo menor que 10');
09     }
10     if ( prime.length > 5 || parseInt(segundo) >= 1000 ) {
11         alert('el primero es mas largo de 5 caracteres o el segundo es igual o
            mayor de 1000');
12     }
```

58 - Hacer no editable un input:

En html:

```
00     <fieldset>
01     <label for="fuerza">Fuerza</label>
02     <input type="text" name="fuer" id="fuerza" readonly="readonly">
03     </fieldset>
```

59 - Variables todo a mayúsculas - Minúsculas:

```
00     convalidada = prompt('¿tienes convalidada EIE? [responde con si o
            no]',convalidada);
01     switch (convalidada.toLowerCase()){ // a minusculas
02     switch (convalidada.toUpperCase()){ // a mayusculas
```

Anexo 02

[Métodos avanzados con Arrays]

A la hora de trabajar con arrays (independientemente del tipo que sean) en JavaScript, nos podemos ver obligados a realizar consultas puntuales. Para ello disponemos de 3 métodos.

Método 'filter'	Si queremos encontrar todos los elementos que cumplan con una condición específica.
Método 'find'	Si queremos saber si al menos uno de los elementos cumple una función específica.
Método 'includes'	Si queremos saber si el array contiene un valor específico.
Método 'indexOf'	Si queremos encontrar el valor del puntero, de un elemento en particular dentro del array.

60.01 Método 'filter':

Podemos usar el método `Array.filter()` para encontrar los elementos dentro de un array que cumplan con cierta condición.

```
00   let arreglo = [10, 11, 3, 20, 5];
01   let mayorQueDiez = arreglo.filter(element => element > 10);
02   alert(mayorQueDiez) // [11, 20]
```

Este código nos mostraría todos los valores que son superiores a '10' en un dialogo, alert.

Importante:

Si no se encuentra ningún valor que este dentro de los criterios del filtro, el filtro devolvería un valor **'vacío'**.

60.02 Método 'find':

Este método `Array.find()` Buscará concorde al filtro que le indiquemos hasta que localice un elemento que cumpla con la condición (elemento que nos devolverá) es importante destacar que, en caso de localizar una coincidencia, se 'detiene' no continúa buscando en los registros.

```
00   let arreglo = [10, 11, 3, 20, 5];
01   let existeElementoMayorQueDiez = arreglo.find(element => element >
02   10);
02   alert(existeElementoMayorQueDiez) // 11
```

Importante:

Si el método 'find' no localiza ningún elemento que concuerde con el filtro introducido devuelve un valor **'undefined'**.

Anexo 01

[Métodos avanzados con Arrays]

60.03 Método 'includes':

El método `.includes(x,x)` hace lo mismo que el 'find' pero devolviendo un 'true' (si localiza le registro) 0 un 'false' (si no lo encuentra).

Este método cuenta a su vez con dos pequeños parámetros:

```
let incluyeValor = arreglo.includes(valorAEncontrar, aPartirDelIndice)
```

Donde:

Valor a encontrar: Es el valor que estamos buscando.

A partir del índice: Es la posición desde donde queremos buscar (0 seria en todo el array).

```
00 let arreglo = [10, 11, 3, 20, 5];
01 let incluyeDiezDosVeces = arreglo.includes(10, 1);
02 alert(incluyeDiezDosVeces) //false
```

60.04 Método 'indexOf' :

Este método `'indexOf(x,x)'` nos devuelve el valor del puntero que marca la posición en el array, del valor especificado en el filtro.

Al igual que el método 'includes()' también posee dos parámetros:

```
let indiceDeElemento = arreglo.indexOf(elemento, aPartirDelIndice)
```

Donde:

Elemento: Es el valor que estamos buscando.

A partir del índice: Es la posición desde donde queremos buscar (0 seria en todo el array).

Importante:

En caso de no encontrar el valor especificado en el filtro, devuelve `'-1'`.

Tanto el método `includes()` como `indexOF()` usan igualdad estricta (`==`) cuando están buscando dentro de un array. Esto significa que no darían por valida una igualdad similar a esta:

```
[ '4' y 4 = false y -1 ]
```

[08-01-23 21:20]

61- Métodos de trabajo JSON**[09-01-23]****¿Qué son los JSON?**

Los JSON son archivos que contienen información que puede ser importada desde cualquier documento JavaScript. Esta información, posteriormente puede ser empleada en el propio HTML (mediante el método DOM) o en las funciones del JavaScript.

61.01 Creación del .json

El archivo '.json' se crea del mismo modo que los .css o .js ; en el interior de un folder dentro del mismo directorio que el resto de web.

Ejemplo:**Datos.json****61.02 Estructura de datos del .json****Llave de comienzo y llave de final {} :**

La estructura de datos del .json es la de un archivo de texto plano, que comienza con '{' y finaliza con '}' El contenido del archivo está en el interior de esas llaves.

Declaración de los datos:**[Dato simple]**

```
00  {
01    "primer dato" : "valor del dato",
02    "segundo dato" : 100,
03    "tercer dato"  : true
04  }
```

Cada uno de los datos está finalizado con una ',' todos menos el último de los datos.

Los datos siempre van en pares:

Propiedad: valor

Declaración de los datos:**[Arrays]**

```
{  "nombre": "coco",
  "apellidos": "miaus",
  "lenguajes_conocidos": ["javascript","python","C++"],
  "edad": 45
}
```

61.02 Estructura de datos del .json**[Continuación]****Declaración de los datos:****[Arrays con objetos]**

```

00    { "pago_agua":[
01      {"id": 0,
02        "fecha": "abril 2022",
03        "pagado": true},
04      {"id": 1,
05        "fecha": "mayo 2022",
06        "pagado": true},
07      {"id": 2,
08        "fecha": "junio 2022",
09        "pagado": true}
10    ]
11  }

```

Pago_agua[0].id	Pago_agua[0].fecha
Pago_agua[0].pagado	

Pago_agua[1].id	Pago_agua[1].fecha
Pago_agua[1].pagado	

Pago_agua[2].id	Pago_agua[2].fecha
Pago_agua[2].pagado	

Importante:

La estructura que se escoja en el primer objeto del array en este método, ha de mantenerse. Si no se hace de este modo, el array o no funcionará o dará un error.

Sintaxis array con objetos:**Nombre_array[****{“nombre atributo_objeto”: valor, nombre atributo_objeto”: valor},****{“nombre atributo_objeto”: valor, nombre atributo_objeto”: valor}****]**

Al igual que con la declaración de datos simples en .Json, el ultimo objeto del array no termina en ‘,’

61.03 Carga del .json**[En archivo de texto]**

Dado que estamos ejecutando javascript en un navegador, la carga del archivo, se hace desde el propio navegador. Para ello hay que emplear un método asíncrono (SYNC) o lo que es lo mismo; un método que carga la página 'instantáneamente' en el momento de solicitarlo, y no continua con la ejecución de la función de carga hasta que se asegura que la carga ha sido correcta (del mismo modo que el método 'promise').

```

00  async function cargadatos(){
01      const response = await fetch ("http://127.0.0.1:5500/datos.json");
02      const json = await response.text();
03      alert(json);
04  }
05  Cargadatos();

```

En este ejemplo podríamos ver el contenido de los datos del .json, en un mensaje emergente (alert) cual archivo de texto.

Sintaxis:

```
const response = await fetch ("http://127.0.0.1:5500/datos.json");
```

Cargamos el contenido de la página "datos.json" en la variable "response"

```
const json = await response.text();
```

Cargamos el contenido de la anterior carga en la variable 'json' en método texto '.text()'

```
alert(json);
```

Mostramos el contenido de la variable json en un alert.

61.04 Carga del .json**[En variables]**

Este método es el útil a la hora de gestionar webs dinámicas (próximo apartado), al mismo tiempo también nos sirve para poder cargar contenidos 'fijos' en páginas web.

> Acceso y carga de un dato Json a variable JavaScript:

```

01  const response = await fetch ("http://127.0.0.1:5500/datos.json");
02  const json = await response.text();
03  let apellido = json.apellido;

```

Se accede al valor del dato, mediante la variable que contiene los datos Json seguido de un punto y el nombre de la variable simple.

61.04 Carga del .json**[En variables]****>Acceso y carga de un array Json a variable JavaScript:**

```

01    const response = await fetch ("http://127.0.0.1:5500/datos.json");
02    const json = await response.text();
03    let apellido = json.lenguajes_conocidos[1];

```

Se accede al valor del dato, mediante la variable que contiene los datos Json seguido de un punto y el nombre del array junto al puntero que corresponda al dato que se quiere emplear.

>Acceso y carga de un objeto de array Json a variable JavaScript:

```

01    const response = await fetch ("http://127.0.0.1:5500/datos.json");
02    const json = await response.text();
03    let apellido = json.pago_agua[0].fecha;

```

Se accede al valor del dato, mediante la variable que contiene los datos Json seguido de un punto y el nombre del array junto al puntero que corresponda al registro del objeto del array, seguido de un punto indicando el atributo del objeto al que se quiere acceder (dentro de ese registro).

61.05 Carga de objeto a Json**[Inversa]**

Existe un método para transferir un objeto de java script a estructura de Json. Este método no registra ni modifica en absoluto el Json que podamos tener en nuestra web, solo transforma el objeto que le transfiramos en "estructura" Json.

Método:

```

00    function Alumno(aw,sox) {
01        this.aw = aw;
02        this.sox = sox;
03    }
04    let coco = new Alumno(1,2);
05    async function obtenerDatos2(){
06        alert(JSON.stringify(coco));
07    }
08    obtenerDatos2();

```

Este sería el constructor, que define el objeto "alumno". Este objeto tiene dos atributos (aw y sox) y ningún método.

Se crea un nuevo objeto "alumno" (Coco)

Por último se transfiere el objeto "coco" al método JSON mediante .stringify()

62- Métodos de trabajo Ajax**[25-01-23]**

En el presente curso no entra Ajax, pero el profesor nos obliga a manejar la función de “promesa” y ‘fetch’ asincrónico mediante Ajax. Por ello, únicamente expondré a continuación el método de trabajo, la única información que me ha sido facilitada.

Diferencias del método Ajax de fetch respecto a ‘.json’

La principal diferencia es que este método carga y maneja los datos ‘a tiempo real’ de modo que, si se combina con el DOM de JavaScript, podemos gestionar contenidos web dinámicamente.

62.01 Carga de los datos Json en Ajax**[Sincrónico]**

```

00  function carga_datos() {
01      // método alternativo, que te permite ver el contenido del json en
02      // el navegador: (clicando en el)
03      //fetch('http://127.0.0.1:5500/2_evaluacion/ejercicio_java_script_J
04      //SON_AJAX/datos/jugadores.json')
05      fetch('./datos/datos.json')
06
07      .then(pre_carga => pre_carga.json())
08
09      .then((datos) => {
10          crear_pagina(datos);
11
12          // debug :
13          // alert('prueba de carga: ' + " " + datos[0].nombre);1
14
15      })
16      .catch(error=>console.log("Error de procesado",error));
17
18
19
20  }
21  Carga_datos();

```

La llamada a función ‘crear_pagina(datos)’ es la que traspasa el contenido del json a la función DOM.

Se tiene que auto ejecutar la función desde la primera vez que se cargue el archivo JavaScript.

La posición del invocador depende de las funciones de la página.

Importante:

Para la correcta carga del fetch, hay que asegurarse que el .json está correctamente linkeado al java script. Para ello se puede realizar un debug mediante ‘alert’ o una carga mediante el loopback de la máquina. Ambos métodos están comentados en la función de carga referenciada.

62- Métodos de trabajo Ajax

[Continuación]

62.02 Datos Ajax sincrónico para DOM JavaScript

La siguiente función es la única referencia que nos han facilitado en el curso de AW de grado medio, para importar e incrustar datos json procedentes de un Ajax en DOM.

```

01  function crear_pagina(empleados){
02      let contenido = document.querySelector('#aquí');
03      contenido.innerHTML = '';
04      for (let empleado of empleados ) {
05          contenido.innerHTML += `
06              <h1>Empleado</h1> <text id="e_empleado">${empleado.numero}</text>
07              <label for="nombre">Nombre</label>
08              ${empleado.nombre}
09              <label for="apellidos">Apellidos</label>
10              ${empleado.apellidos}
11              <label for="cargo">Cargo</label>
12              ${empleado.cargo}
13              
14              `
15          }
16      };
17  }

```

Selector del área DOM por ID, y vaciado previo a carga.

Bucle que se repite tantas veces como registros posea el json importado por Ajax.

Para incrustar los datos del json en el DOM solo tenemos que referenciarlos indicando el nombre de la variable que hemos asignado arbitrariamente en el bucle for, y un punto. Tras este punto, el nombre del campo que deseamos importar del registro json.

Acceso dato normal:

`${empleado.nombre}`

Acceso a dato en array:

`${empleado.idiomas[0]}`

Acceso a método:

`${empleado.pluses()}`

El método solo devuelve valor si su return() está correctamente declarado.

Importante:

Este método funciona a base de “promesas” ello quiere decir que sigue un orden escalonado que obliga a seguir una ejecución secuencial. Si no se finaliza la primera promesa, la función, cambia por completo, aunque prosigue su ejecución. Las variables poseen un scope local, o lo que es lo mismo; es indiferente el nombre o significado que tengan. Su asignación es totalmente arbitraria e insignificante fuera de la propia función. Aun así es importante conocer su punto de referencia, de lo contrario será imposible acceder a los datos del .json

62- Métodos de trabajo Ajax**[Continuación]****62.03 Filtro de carga mediante 'dropdown' HTML**

Este método de trabajo permite discriminar los contenidos de carga del DOM filtrando los contenidos mediante una variable de un dropdown.

En HTML:

```
00 <div><h1>Ejercicio_ajax_03</h1></div>
01 <div id="filtro">
02 <select id="Salario">
03 <option value="Todos">Todos</option>
04 <option value="350 €">350 €</option>
05 <option value="650 €">650 €</option>
06 </select>
07 </div>
```

Importante:

El valor que vamos a emplear para filtrar la carga del DOM es el correspondiente al campo "value" del dropdown.

Este campo SOLO PUEDE TENER VALOR STRING, puesto que el HTML no permite valores de atributo de etiqueta sin comillas.

En JavaScript:

[01 de 02]

En este ejemplo empleamos el mismo código de carga que en el 10.02, pero modificado para poder mostrar solo los registros que coincidan con el valor seleccionado en el dropdown de la página HTML.

```
00 function crear_pagina(empleados){
01   let contenido = document.querySelector('#aquí');
02   contenido.innerHTML = '';
03   var filtro_sueldo = document.getElementById("Salario").value;
04   for (let empleado of empleados ) {
05     if ((filtro_sueldo == "Todos") || ((empleado.sueldo.indexOf(filtro_sueldo)) == 0))
06       {
07         contenido.innerHTML += `
08       } }`;
09   }
```

Asignamos el valor del dropdown a la variable 'filtro_sueldo'

El registro del que extraemos los datos del DOM es el correspondiente al que posea el mismo valor que la variable 'filtro_sueldo' en su campo 'sueldo'.

Si no se declara ninguno, se mantiene el bucle for al completo (con todos los valores del json), esto corresponde a "todos".

62- Métodos de trabajo Ajax**[Continuación]****En JavaScript:****[02 de 02]**

Para poder emplear correctamente el dropdown tenemos que incluir un evento “change” de modo que cada vez que se modifique el valor del desplegable, los datos vuelvan a cargarse en la página (actualizándose de este modo en base al filtro).

```
01    carga_datos();  
02    document.getElementById('Salario').addEventListener('change', () => {  
03    carga_datos();  
04    });
```

El código sobre estas líneas ha de estar en la cabecera del .js , de este modo nada más cargarse el archivo de scripts se cargan los datos DOM y tras esto, el navegador queda a la escucha del evento “cambio” en el desplegable. Si cambia su valor, vuelve a cargar los datos DOM de la página.

Estructura del JavaScript:

Esta sería la estructura correcta para poder manejar correctamente el dropdown sin errores de sincronización.

- 1- **[Carga]** datos del json para el DOM

- 2- Escucha si hay cambios en el desplegable de filtro, si es así
Vuelve a cargar los datos.

- 3- Utiliza los datos del json en DOM, creando la sección del HTML referenciada.

- 4 – **[Carga]** los datos siguiendo las instrucciones del fetch, tras esto (si se cumple la promesa) inicia la construcción de la página en base a los datos del json y el valor del filtro desplegable.

Al iniciarse la carga del archivo ‘.js’ por primera vez, se cargan los datos del json, seguidamente se crea la sección del HTML en base a los datos del json por DOM. Por último, se mantiene a la escucha por si cambia el valor del desplegable.

Importante:

Por trivial que aparente ser, es vital tener muy claro el orden de ejecución de cada una de las funciones. De lo contrario el filtrado no se dará correctamente.

62- Métodos de trabajo Ajax**[Continuación]****62.04 Actualización de carga del src .js**

Es posible cargar los bloques de scripts en el header del documento HTML, para ello solo tenemos que incluir el parámetro 'defer = defer' en el interior de la etiqueta.

En el html:

```
00    <!DOCTYPE html>
01    <html lang="en">
02    <head>
03    <meta charset="UTF-8">
04    <meta http-equiv="X-UA-Compatible" content="IE=edge">
05    <meta name="viewport" content="width=device-width, initial-scale=1.0">
06    <link rel="stylesheet" href="css/style.css">
07    <link rel="stylesheet" href="css/sweetalert2.min.css">
08    <script src="js/sweetalert2.all.min.js" ></script>
09    <script src="js/script.js" defer="defer" ></script>
10    <title>Ejercicios_ajax_02</title>
11    </head>
```

62.05 Actualización de sistema de datos en json

Es posible (y muy recomendable) crear los json mediante la estructura de array, como en el ejemplo bajo estas líneas.

```
00    [
01    {"numero" : 1, "cargo" : "reponedor",
02    "sueldo" : "350 €"
03    },
04    {"numero" : 2, "cargo" : "cajero",
05    "sueldo" : "750 €"}
06    ]
```

[27-01-23 1:03]

63- Método 'Promise' Javascript**[Ultimo anexo]****¿Qué es un método 'promise'?**

El método promesa en javascript, funciona de un modo parejo al que lo haría la unión de un condicional simple (if + else) y una función constructor.

Al ejecutar una promesa se comprueba una expresión o método, a razón del resultado del mismo se continua hacia una de dos funciones:

- Si se ejecuta correctamente la expresión / método **[Success]**
- Si falla la expresión / método **[Reject]**

Declaración de la promesa

Las promesas se declaran del mismo modo que las funciones constructor (o plantillas de objeto).

Ejemplo:

```
00  ejemplo_promesa.then(  
01    function(ok){ código si se cumple},  
02    function(error){ código si no se cumple}  
03  );
```

Ejecución de la promesa

Al ejecutar la promesa, se declara del mismo modo que los objetos producto del constructor, con la salvedad que se emplean dos funciones a modo de parámetros.

Ejemplo:

```
00  let ejemplo_promesa = new promise(function(myresolve,myreject){  
01    myresolve();  
02    myreject();  
03  });
```

Tal como está aquí mostrado sería erróneo. Solo puede haber una resolución; 'resolve' o 'reject'. Esto lo podemos seleccionar mediante un condicional o manualmente.

Funcionamiento de la promesa

Al ejecutar la promesa creamos un condicional o expresión que ha de evaluarse (también puede ser una carga json o similar) al cumplirse o no, se ejecuta el código correspondiente a la resolución o al fallo presente en la declaración de la promesa.

Importante:

El nombre de la promesa ha de ser el mismo tanto en la declaración como en la ejecución. 'new promise' no es una variable ni función, es un nombre reservado, corresponde a la promesa.

64- Método DOM producto de constructor y objetos**[Penúltimo examen]**

Existen dos métodos a la hora de cargar material en HTML mediante javascript.

[Método 01 de DOM]**-Sin constructor ni objetos-**

Este es el que anteriormente hemos visto, consistiría en identificar el 'id' de la etiqueta y posteriormente (gracias a este) introducir un contenido HTML mediante DOM.

Ejemplo:

```
01 document.getElementById('aqui').innerHTML = "<h1 id='titulo' style='color:
02 aquamarine;'>Esto es un ejemplo</h1>";
```

Cuando se emplea el 'innerHTML' se recomienda incrustar el style en la propia etiqueta. Todos los estilos y parámetros de etiqueta se declaran en este método mediante comillas simples "

[Método 02 de DOM]**-Con constructor y objetos -**

En este caso primero creamos el constructor, seguidamente los objetos y por último el constructor de la página o contenido.

Constructor:

```
00 function ofertas(id_del_objeto,linea_sup, linea_infe,detalles,
01 precio_original, precio_total){
02 this.id_00 = id_del_objeto;
03 this.descriptivo_01 =linea_sup;
04 this.descriptivo_02 =linea_infe;
05 this.detalle_00 =detalles;
06 this.precio_org =precio_original;
07 this.precio_total =precio_total;
08 }
```

[Método 02 de DOM]

-Con constructor y objetos -

En este caso creamos 4 objetos en base al constructor, asignándolos posteriormente a un array (imprescindible para automatizar la carga más tarde)

Objetos:

```
01 let oferta_01 = new ofertas ("obj_01","cuspito de la marea","muy bonito el...","Cerca
    de no se donde, Km 162",150,80);

02 let oferta_02 = new ofertas ("obj_02","jusjus justi","muy boni;ito el...","Cerca de
    no se donde, Km 1882",10,80);

03 let oferta_03 = new ofertas ("obj_03","la marea","muy lalalala el...","Cerca de
    njottt nde, Km 1792",550,180);

04 let oferta_04 = new ofertas ("obj_04","UFF lalala","muy bonito el...","Cerca de no
    se donde, Km 162",160,90);

05 let array_ofertas = [oferta_01,oferta_02,oferta_03,oferta_04];

06 construye(array_ofertas);
```

Constructor de página :

```
00 function construye(tope){
01 let contenido = document.querySelector('#contenedor');
02 contenido.innerHTML = '';
03 for( n of tope){
04 contenido.innerHTML += `
05 <div id="${n.id_00}" style="width:21vw; margin-right: 1.5vw;">
06 
07 <h3 id="desc_sup" >${n.descriptivo_01} <br> ${n.descriptivo_02}</h3>
08 <p id="desc_infe">${n.detalle_00}</p>
09 </div>
10 `
11 }
12 }
```

La variable contenido pasa a tener todo el contenido de la sección 'contenedor' del HTML.

document.querySelector

Selecciona la sección del HTML

For (n of tope)

'n' hereda el array y se repite bucle hasta que se llega al final de registros de 'n'

Contenido.innerHTML += ``

introduce el código HTML en la seccion